

Module 15: G-Code Standards and Post-Processing

Section 15.1 - Introduction to G-Code Standards and Post-Processing

Overview

G-code (officially RS-274D/ISO 6983) is the universal programming language that bridges human design intent and machine motion. First developed in the 1950s at MIT as part of early numerical control research, G-code has evolved into the lingua franca of CNC manufacturing, enabling everything from simple 2D profiles to complex 5-axis sculptured surfaces.

This module provides comprehensive coverage of G-code standards, programming methodologies, post-processing techniques, and control system dialects. Whether you're hand-coding precision parts or configuring CAM post-processors, understanding G-code at a fundamental level is essential for CNC engineering mastery.

The Role of G-Code in Modern Manufacturing

From CAD to Chips

The modern CNC workflow follows this path:

CAD Model -> CAM Toolpath -> Post-Processor -> G-Code -> Machine Controller -> Physical Part

G-code sits at the critical junction between digital design and physical reality. It translates geometric intent into precise machine commands:

- **Motion control:** Linear moves, arcs, helical interpolation
- **Process parameters:** Feed rates, spindle speeds, coolant activation
- **Tool management:** Tool changes, length offsets, radius compensation
- **Coordinate systems:** Work offsets, fixture setups, part alignment
- **Logic and flow:** Conditional branching, loops, macros

Why G-Code Still Matters

Despite advances in conversational programming and graphical interfaces, G-code remains fundamental because:

1. **Universality:** Every CNC control system ultimately executes G-code or a variant
2. **Precision:** Direct control over every motion and parameter
3. **Debugging:** Understanding G-code enables troubleshooting CAM output
4. **Optimization:** Hand-editing can improve cycle times and surface finish
5. **Custom operations:** Probing, in-process measurement, adaptive machining
6. **Legacy systems:** Decades of proven programs still in production use

Historical Context

Evolution of Numerical Control

- **1952:** MIT Servomechanisms Lab develops first NC milling machine
- **1960s:** EIA RS-274 standard emerges, “G” and “M” code convention established
- **1970s:** ISO 6983 international standard published
- **1980s:** CNC controls add macros, variables, canned cycles
- **1990s:** High-speed machining drives look-ahead and smoothing
- **2000s:** 5-axis and multi-tasking machines expand G-code capabilities
- **Present:** Real-time controls, adaptive feeds, integrated probing

Standards Organizations

- **ISO 6983:** International baseline standard (often called “G-code”)
- **EIA RS-274-D:** American precursor to ISO standard
- **DIN 66025:** German standard, basis for European controls
- **JIS B 6315:** Japanese standard, influences Asian manufacturers

Module Scope and Objectives

What You Will Learn

By completing this module, you will be able to:

1. **Read and write G-code programs** from scratch using standard commands
2. **Understand block structure**, modal vs. non-modal commands, and program organization
3. **Apply motion commands** including rapid positioning, linear interpolation, and circular arcs
4. **Manage coordinate systems** with work offsets, absolute/incremental modes, and tool compensation
5. **Utilize auxiliary functions** for spindle, coolant, tool changes, and program flow
6. **Implement canned cycles** for drilling, tapping, boring, and pocketing operations
7. **Follow best practices** for safe, efficient, maintainable program development
8. **Configure post-processors** to translate CAM output for specific machine controls
9. **Use advanced features** including macros, variables, parametric programming, and subprograms
10. **Navigate control dialects** across FANUC, Siemens, Heidenhain, LinuxCNC, and others
11. **Verify programs** using simulation, dry-run techniques, and debugging tools
12. **Integrate G-code knowledge** with mechanical design, electronics, and process planning

Prerequisites

This module assumes familiarity with:

- **Module 1-2:** Mechanical systems and axis nomenclature (X, Y, Z, A, B, C)
- **Module 3:** Linear motion systems and positioning accuracy
- **Module 4:** Control electronics and stepper/servo motor operation
- **Module 14:** LinuxCNC HAL architecture (useful but not required)
- **Basic mathematics:** Trigonometry, coordinate geometry, vector calculations
- **CAD/CAM concepts:** Toolpaths, cutting tools, machining operations

Module Structure

This module is organized into twelve sections:

1. **Introduction** (this section) - Context and objectives
2. **G-Code Structure** - Block format, address codes, syntax rules
3. **Motion Commands** - G00, G01, G02, G03, feed rate control
4. **Coordinate Systems** - Work offsets, absolute/incremental, tool compensation
5. **Auxiliary Functions** - M-codes for spindle, coolant, tool changes

6. **Canned Cycles** - G81-G89 drilling, tapping, boring sequences
7. **Programming Best Practices** - Structure, safety, optimization, documentation
8. **Post-Processing** - CAM integration, post-processor configuration
9. **Advanced Features** - Macros, variables, parametric programming
10. **Control System Dialects** - FANUC, Siemens, Heidenhain, LinuxCNC variations
11. **Simulation and Verification** - Toolpath checking, DNC, validation techniques
12. **Conclusion** - Summary, integration, next steps

Real-World Applications

CNC Milling

G-code programs for milling machines control: - 3-axis contouring for complex 2.5D parts - 4/5-axis simultaneous machining for turbine blades, impellers - High-speed finishing with optimized feed rates - Adaptive clearing with force feedback

CNC Turning

Lathe G-code includes: - Facing, turning, threading cycles - Contour turning with radius compensation - Live tooling and C-axis milling - Sub-spindle transfer operations

Plasma, Laser, Waterjet

2D cutting systems use simplified G-code: - G01 linear cuts with kerf compensation - G02/G03 arcs for rounded corners - Pierce delays and lead-in/lead-out moves - Nesting and common-line cutting

Additive Manufacturing

FDM 3D printers use G-code dialects: - Extruder control (E-axis moves) - Temperature management (M104, M109) - Layer-by-layer deposition - Retraction and wipe sequences

Hybrid and Multi-Process

Advanced machines combine operations: - Mill-turn centers with live tooling - Additive + subtractive hybrid systems - Waterjet + laser combination tools - Pick-and-place + machining cells

Key Takeaways

1. G-code is the **universal language** that translates design intent into machine motion
2. Understanding G-code enables **debugging, optimization, and customization** beyond CAM software
3. **Standards** (ISO 6983) provide a common baseline, but **dialects** vary by manufacturer
4. G-code combines **simplicity** (ASCII text, block structure) with **power** (3D motion, logic, variables)
5. **Safety** is paramount: always simulate and verify before running new programs
6. This module builds on mechanical, electronic, and control system knowledge from previous modules
7. Mastering G-code is essential for **complete CNC engineering competency**

Next: Section 15.2 - G-Code Structure

Previous: Module 15 Overview

Section 15.10 - Control System Dialects

Overview

While ISO 6983 provides a baseline G-code standard, major CNC control manufacturers have developed distinct dialects with unique syntax, features, and conventions. Understanding these differences is essential for adapting programs across machines, configuring post-processors, and troubleshooting control-specific behavior.

This section compares major control systems—FANUC, Siemens, Heidenhain, Haas, Mazak, and LinuxCNC—highlighting syntax differences, unique features, and practical adaptation strategies.

ISO 6983 Baseline Standard

Common Ground Across All Controls

All major control systems share these fundamental concepts:

Core motion commands: - G00 (rapid), G01 (linear feed), G02/G03 (circular interpolation) - X, Y, Z axis designations - F (feed rate), S (spindle speed)

Basic structure: - Block-based execution (one line at a time) - Modal vs. non-modal commands - Coordinate systems (absolute/incremental)

Tool management: - T-word for tool selection - M06 for tool change (most controls)

Program control: - M-codes for spindle, coolant, program flow

Where Dialects Diverge

Syntax variations: - Leading zeros (G01 vs. G1) - Decimal places - Comment delimiters - Line numbering formats

Feature availability: - Canned cycles (different cycle numbers) - Macro/variable syntax - Subprogram calls - Tool compensation methods

Machine-specific M-codes: - Manufacturer-specific auxiliary functions - Proprietary features

FANUC Control Dialect

Overview

FANUC is the most widely used CNC control globally, forming the de facto standard for industrial CNC programming.

Market presence: - ~50% global market share - Standard on Haas, Doosan, Okuma, DMG Mori, many others - Most CAM post-processors default to FANUC syntax

Syntax Characteristics

Leading zeros:

G01 (Preferred, but G1 also accepted)
M03 (Preferred, but M3 also accepted)

Decimal format:

X10.5 (Decimal point required)
X10.0 (Trailing zero recommended)

Comments:

(COMMENT IN PARENTHESES)

Program structure:

```
%  
O1234  
(PROGRAM NAME)  
G21 G90 G17  
G54  
...  
M30  
%
```

FANUC-Specific Features

Subprogram calls:

```
M98 P8500      (Call local subprogram O8500)  
M98 P8500 L5   (Call O8500, repeat 5 times)  
M99           (Return from subprogram)
```

Custom M-codes (common on FANUC machines):

```
M19           (Spindle orientation)  
M41-M44       (Gear range selection)  
M60           (Pallet change)
```

Macro variables:

```
#1 = 10.5      (Variable assignment)  
#100 = #5021   (Read system variable)
```

FANUC Macro B (parametric programming):

```
IF [#1 GT 10] THEN #2 = 20  
WHILE [#1 LT 100] DO 1  
  #1 = #1 + 1  
END 1
```

FANUC Variants

FANUC 0i/16i/18i/21i: - Standard FANUC dialect - Most common in modern machines

FANUC 30i/31i/32i: - Enhanced features - Faster processing - More memory

FANUC Custom Macro: - Variable support (#1-#999) - Conditional logic - Loops and arithmetic

Siemens Control Dialect

Overview

Siemens (Sinumerik) controls are prevalent in European machines and high-end machining centers.

Market presence: - Strong in Europe (Germany, Switzerland, Italy) - Common on DMG Mori, Chiron, Hermle, Starrag

Syntax Characteristics

No leading zeros:

```
G1           (Not G01)  
M3           (Not M03)
```

Decimal format:

X10.5 (Same as FANUC)

Comments:

; COMMENT WITH SEMICOLON

Program structure:

```
; PROGRAM NAME
G54
G17 G90 G94
...
M2 (M2 more common than M30)
```

Siemens-Specific Features

Subprogram calls:

```
CALL "SUBPROG.SPF" (Call external subprogram by name)
M17 (Return from subprogram)
```

R parameters (variables):

```
R1 = 10.5 (Variable assignment)
R2 = R1 + 5 (Arithmetic)
X = R1 Y = R2 (Use in motion)
```

Conditional logic:

```
IF R1 > 10
  R2 = 20
ENDIF
```

Loops:

```
REPEAT P1
  G1 X = X + 10
ENDLABEL P1
```

Siemens Cycles:

```
CYCLE81(10, 0, 2, -20) (Drilling cycle with parameters)
MCALL CYCLE81 (Activate cycle)
X10 Y10 (Drill at position)
MCALL (Cancel cycle)
```

Siemens 840D/828D Specifics

ShopMill/ShopTurn: - High-level conversational programming - Generates G-code internally

Synchronized actions:

```
SYNFCT (Synchronized function)
```

Transformations:

```
TRANS X10 Y20 (Translation)
ROT RPL = 45 (Rotation)
```

Heidenhain Control Dialect

Overview

Heidenhain (TNC) controls emphasize conversational programming with clear, English-like syntax.

Market presence: - Common in Europe, especially Germany - High-end machine tools - Popular in tool and die shops

Syntax Characteristics

Conversational format:

```
0 BEGIN PGM PART1 MM      (Program start, metric units)
1 TOOL CALL 5 Z S3000      (Tool 5, spindle 3000 RPM)
2 L X+10 Y+20 R0 F500 M    (Linear move to X10 Y20)
3 CC X+50 Y+50             (Circle center at X50 Y50)
4 C X+60 Y+50 DR+ R10 F300 M (Circular move)
...
99 END PGM PART1          (Program end)
```

Characteristics: - Line numbers at start of each block - Explicit commands (L for linear, C for circular) - DR+ / DR- for CW/CCW direction - R0/R/RF for approach behavior

Heidenhain-Specific Features

Cycle definitions:

```
CYCL DEF 200 DRILLING      (Define drilling cycle)
  Q200=2                   (Safety clearance)
  Q201=-20                 (Depth)
  Q206=250                 (Feed rate)
CYCL CALL                  (Activate cycle)
L X+10 Y+10 M99            (Drill at position)
```

Subprograms:

```
CALL LBL 1                 (Call label 1)
...
LBL 1                      (Label definition)
...
LBL 0                      (End of subprogram)
```

FK Free Contour Programming: - Describes part geometry directly - Controller calculates toolpath

Q parameters (variables):

```
FN1: Q1 = 10.5             (Assign to Q parameter)
FN2: Q2 = Q1 + 5           (Arithmetic)
L X+Q1 Y+Q2                (Use in motion)
```

Heidenhain vs. ISO Mode

Heidenhain controls support **ISO mode** for compatibility:

```
BEGIN PGM ISO_MODE MM
G01 X10 Y20 F500          (Standard ISO syntax)
...
END PGM ISO_MODE
```

Most programmers use Heidenhain's native conversational format for clarity.

Haas Control Dialect

Overview

Haas uses FANUC-based controls with manufacturer-specific customizations.

Market presence: - Largest CNC machine builder in North America - Common in job shops, schools - FANUC-compatible with Haas extensions

Syntax Characteristics

Generally FANUC-compatible:

G01 X10 Y20 F500 (Standard FANUC syntax)
M03 S2000 (Spindle on)

Haas-specific M-codes:

M12 (Thru-spindle coolant on)
M13 (Spindle on CW + coolant)
M26 (Water blast on)
M95 (Sleep mode - waiting)
M96 (Jump if no input)
M109 (Z-axis brake on)

Haas-Specific Features

Macro programming:

#1 = 10.0 (FANUC-style variables)
IF [#1 GT 5] GOTO 100 (Conditional jump)

Visual Quick Code (VQC): - Haas conversational programming - Generates G-code for common operations - Similar to FANUC Manual Guide

Advanced geometry options:

G187 (Smooth acceleration)
G05.1 Q1 (AI contour control)

Wireless probing (some models):

G65 P9832 (Probing macro call)

Mazak Control Dialect

Overview

Mazak machines use proprietary Mazatrol conversational programming or ISO G-code.

Market presence: - Major Japanese manufacturer - Multi-tasking and turn-mill specialists - Conversational programming focus

Mazatrol Conversational

Unit-based programming:

```
UNIT 1: FACE
  Z-START: 0
  Z-END: -5
  FEED: 0.2
END UNIT
```

Advantages: - No G-code knowledge required - Graphical programming - Fast for simple parts

Disadvantages: - Not portable to other machines - Limited for complex geometry

Mazak ISO Mode

Mazak controls also support ISO G-code (EIA/ISO mode):

```
00001
G54 G90 G00 X0 Y0
M03 S2000
G01 Z-5.0 F200
...
M30
```

Syntax similar to FANUC with Mazak-specific M-codes:

```
M60                (Pallet change)
M205               (Tailstock advance)
M78                (Sub-spindle interlock on)
```

LinuxCNC Dialect

Overview

LinuxCNC is an open-source CNC control running on PC hardware with real-time Linux kernel.

Market presence: - Open-source community - Hobbyists, researchers, custom machines - Highly customizable

Syntax Characteristics

Generally FANUC-compatible:

```
G01 X10 Y20 F500      (Standard syntax)
```

Comment styles:

```
(PARENTHESIS COMMENT)
; SEMICOLON COMMENT
# HASH COMMENT (some versions)
```

Case insensitive:

```
G01 X10 Y20           (Same as...)
g01 x10 y20
```

LinuxCNC-Specific Features

O-word programming (structured):

```
0100 IF [#1 GT 10]
    (statements)
0100 ELSE
    (statements)
0100 ENDIF
```

```
0200 WHILE [#1 LT 100]
    #1 = [#1 + 1]
0200 ENDWHILE
```

```
0300 DO [#1 = 1, 10, 1]
    (loop body)
0300 ENDDO
```

Subprograms:

```
0100 CALL          (Call 0100 subprogram)
0100 SUB           (Subprogram start)
...
0100 ENDSUB        (Subprogram end)
```

Digital I/O control:

```
M64 P0            (Set digital output 0)
M65 P0            (Clear digital output 0)
M66 P0 L1         (Wait for digital input 0)
```

HAL integration: - Custom M-codes via shell scripts - Direct hardware access - Real-time pin control

Named parameters:

```
#<diameter> = 10.0
#<radius> = [#<diameter> / 2]
G01 X#<radius> F500
```

LinuxCNC Advantages

Flexibility: - Custom kinematics - Custom M-codes - Tool change scripts - Probing routines

Cost: - Free and open-source - Runs on standard PC hardware

Control Dialect Comparison Table

Feature	FANUC	Siemens	Heidenhain	Haas	LinuxCNC
Leading zeros	G01	G1	N/A	G01	G01 or G1
Comment style	()	;	() ;	()	() ;
Subprogram call	M98 P___	CALL	CALL LBL	M98 P___	O___ CALL
Variable prefix	#	R	Q	#	#
Conditional	IF-THEN	IF-ENDIF	IF-ENDIF	IF-GOTO	O___ IF
Loop	WHILE-DO	REPEAT	REPEAT	WHILE	O___ WHILE/DO
Program end	M30	M2	END PGM	M30	M30 or M2
Arc format	I/J/K or R	I/J/K	CC + DR	I/J/K or R	I/J/K or R
Conversational	Manual Guide	ShopMill	TNC Dialog	VQC	None (external)

Adapting Programs Between Controls

FANUC to Siemens

Changes required:

```
(FANUC)                (SIEMENS)
G01 -> G1
M03 -> M3
M98 P5000 -> CALL "O5000.SPF"
#1 = 10 -> R1 = 10
```

(COMMENT) -> ; COMMENT
M30 -> M2

FANUC to Heidenhain

Significant rewrite required:

(FANUC)	(HEIDENHAIN)
O1234 -> O	BEGIN PGM PART1 MM
G01 X10 Y20 F500 -> 1 L	X+10 Y+20 R0 F500 M
G02 X20 Y20 I5 J0 -> CC	X+15 Y+20
	C X+20 Y+20 DR+ R5 F500 M
M30 -> END	PGM PART1

Universal G-Code Practices

For maximum portability:

1. **Stick to ISO 6983 basics:**
 - G00, G01, G02, G03
 - Standard work offsets (G54-G59)
 - Common M-codes (M03, M05, M08, M09, M30)
2. **Avoid control-specific features:**
 - Custom cycles
 - Proprietary macro syntax
 - Manufacturer M-codes
3. **Document dialect:**
 - Note which control the program was written for
 - Include conversion notes
4. **Use post-processors:**
 - Let CAM software handle dialect differences
 - Configure post for target control

Key Takeaways

1. **ISO 6983** provides baseline standard; **dialects** add manufacturer-specific features
2. **FANUC** is the most common dialect, forming the de facto standard
3. **Siemens** uses no leading zeros, semicolon comments, R parameters
4. **Heidenhain** uses conversational format with English-like commands
5. **Haas** is FANUC-compatible with custom M-codes
6. **LinuxCNC** is open-source, highly customizable, FANUC-like syntax
7. **Adaptation** between controls requires syntax translation and feature mapping
8. **Post-processors** handle dialect differences automatically in CAM workflows
9. **Portability** maximized by sticking to ISO 6983 basics
10. **Understanding dialects** essential for multi-machine shops and troubleshooting

Next: Section 15.11 - Simulation and Verification

Previous: Section 15.9 - Advanced Features

Section 15.11 - Simulation, Verification, and Program Validation

Overview

Simulation and verification are critical safeguards that prevent costly crashes, tool breakage, and scrapped parts. Virtual machining validates G-code programs before they ever reach physical hardware, enabling detection of collisions, feed rate errors, missing tool offsets, and geometric mistakes in a safe software environment.

This section covers simulation strategies, verification tools, DNC communication, and systematic validation workflows for ensuring program correctness.

Why Simulation is Essential

Risks of Running Unverified Programs

Without simulation: - Tool crashes into fixtures, clamps, or workpiece - Rapid moves executed at cutting depth - Missing feed rates cause stalls or runaway - Wrong work offsets position tool incorrectly - Tool length offsets not applied, causing table crashes - Arc geometry errors create unexpected motion

Consequences: - Broken tools (\$50-\$500+ per tool) - Damaged spindle (\$5,000-\$50,000 repair) - Scrapped parts (\$100-\$10,000+ material and labor) - Machine downtime (hours to days) - Safety hazards (flying debris, crashes)

Benefits of Simulation

Detect errors before machining: - Collision detection - Toolpath verification - Feed rate validation - Work offset verification - Tool length offset confirmation - Cycle time estimation

Optimize programs: - Identify inefficient rapids - Find bottlenecks in cycle time - Visualize tool engagement - Confirm surface finish approach

Training and communication: - Show operators expected toolpath - Document setup requirements - Verify part geometry match

Types of Simulation

1. Toolpath Verification (2D/3D Visualization)

What it does: - Displays programmed toolpath as 3D lines - Shows rapid moves vs. feed moves - Visualizes arcs and helical paths - No material removal simulation

Tools: - CAMotics (free, open-source) - NC Viewer (web-based, free) - G-Wizard Editor - CAM software built-in viewers

Example - CAMotics:

```
camotics program.nc    # Load and visualize G-code
```

Advantages: - Fast (no material calculation) - Easy to spot missing feed rates, wrong positions - Good for quick verification

Limitations: - Doesn't show material removal - Won't detect tool/part collisions in 3D space - Can't verify surface finish quality

2. Material Removal Simulation

What it does: - Simulates actual cutting process - Removes material from virtual stock - Shows remaining material, gouges, excess - Calculates final part geometry

Tools: - CAM software (Fusion 360, Mastercam, HSMWorks) - Standalone simulators (NCSimul, VERICUT, CimcoEdit) - Machine control built-in simulation

Example - Fusion 360 simulation: 1. Setup -> Simulate 2. Select stock model 3. Run simulation, observe material removal 4. Check for collisions, excess material, gouges

Advantages: - Realistic visualization of cutting process - Detects over-cutting (gouges) and under-cutting (excess material) - Verifies clearances between tool and part - Provides accurate cycle time estimation

Limitations: - Slower than toolpath-only verification - Requires accurate stock and fixture models

3. Machine Simulation (Kinematic)

What it does: - Simulates entire machine including spindle, table, fixtures - Detects collisions between tool holder, spindle head, and fixtures - Accounts for machine kinematics (especially 4/5-axis) - Validates work envelope limits

Tools: - VERICUT (industry standard) - NCSimul Machine - CAM software with machine models - Control-built-in simulation (Siemens, Heidenhain)

Example - VERICUT: - Import machine model (geometry, kinematics, limits) - Load G-code program - Run full simulation with collision detection - Review collision report, analyze near-misses

Advantages: - Most comprehensive verification - Essential for complex multi-axis machines - Detects fixture/clamp collisions - Validates rotary axis limits and coordination

Limitations: - Expensive software (\$10,000+) - Requires accurate machine model - Slower simulation time

Simulation Workflow

Step 1: Quick Toolpath Check

Before running detailed simulation:

(Load program in NC Viewer or CAMotics)

1. Verify program loads without syntax errors
2. Check toolpath appears reasonable
3. Confirm rapids and feeds are distinct
4. Verify arcs are smooth (no radius errors)
5. Check Z-height clearances

Look for obvious errors: - Tool starting inside part (work offset wrong) - Rapids at Z=0 (Z-clearance missing) - Single straight line instead of expected contour (feed rate zero) - Unexpected jumps (wrong work offset or coordinate mode)

Step 2: Material Removal Simulation

In CAM software or standalone simulator:

1. **Define stock:**
 - Material dimensions
 - Stock origin relative to part
 - Material type (optional, for force simulation)
2. **Load fixtures (if available):**
 - Vise model
 - Clamps, parallels
 - Workholding
3. **Run simulation:**
 - Play forward at real-time or fast-forward
 - Pause at tool changes

- Inspect part at critical features
4. **Check for errors:**
- **Gouges:** Tool cuts into finished surface (red regions)
 - **Excess material:** Part features not fully cut (blue regions)
 - **Collisions:** Tool/holder hits fixture or part
 - **Out-of-tolerance:** Part geometry exceeds tolerance bands

Step 3: Control-Based Verification

On the machine control (if available):

(Most modern controls have built-in graphics)

1. Load program into control
2. Select "Graphics" or "Verify" mode
3. Run program in simulation (spindle/axes don't move)
4. Watch toolpath trace on screen
5. Verify against part print

Advantages: - Uses actual machine kinematics - Accounts for control-specific G-code interpretation - Tests program compatibility with control dialect

Limitations: - Basic visualization (typically 2D or simple 3D) - No material removal or collision detection

Step 4: Dry Run on Machine

After simulation, before cutting:

(Dry run procedure)

1. Load all tools, measure and set offsets
2. Set work offsets (touch-off on stock or fixture)
3. Load program
4. Enable "Dry Run" or "Single Block" mode
5. Set feed override to 0% or 10%
6. Run program with Z-axis 25-50mm above actual work surface
(Either raise Z-offset or position stock lower)
7. Watch for unexpected motion, verify tool changes work

Critical checks during dry run: - Tool approaches correct XY positions - Rapids clear all fixtures and clamps - Tool changes execute properly - Feed moves occur where expected (not rapids) - Program completes without alarms

Step 5: First Article Inspection

First part with new program:

1. **Reduced speed:** Run at 50% feed override
2. **Single block:** Step through critical sections
3. **Stop and measure:** Check dimensions after roughing, before finishing
4. **Full inspection:** Measure all dimensions, check surface finish
5. **Document issues:** Note any required offsets, program edits

Simulation Tools and Software

Free and Open-Source

CAMotics: - Platform: Windows, Mac, Linux - Features: 3D toolpath, material removal, STL export - Best for: General G-code verification, hobbyists - Download: <https://camotics.org>

NC Viewer: - Platform: Web browser - Features: 3D toolpath visualization, no installation - Best for: Quick checks, sharing with others - URL: <https://ncviewer.com>

LinuxCNC Axis GUI: - Platform: Linux (LinuxCNC) - Features: Real-time preview, backplot, DRO - Best for: LinuxCNC users, real machine control

Commercial CAM Software (Includes Simulation)

Fusion 360 (\$495/year or free for hobbyists): - Integrated CAM and simulation - Material removal, tool library - Cloud-based, cross-platform

Mastercam (\$5,000-\$15,000+): - Industry-standard CAM - Advanced simulation and verification - Backplot, solid verify, toolpath analysis

HSMWorks (SolidWorks add-in, \$5,000+): - Integrated with SolidWorks CAD - Stock simulation, collision detection

Professional Verification Software

VERICUT (\$10,000-\$50,000+): - Industry-leading machine simulation - Full kinematic validation - Collision detection, optimization - Used in aerospace, automotive

NCSimul Machine (\$8,000-\$20,000): - Machine-centric simulation - Virtual machine commissioning - Post-processor validation

CimcoEdit (\$500-\$1,500): - G-code editor with backplot - 2D/3D verification - DNC communication

DNC Communication and File Transfer

Direct Numerical Control (DNC)

DNC enables communication between computers and CNC machines:

RS-232 serial (legacy):

Baud rate: 9600, 19200 (machine-dependent)

Data bits: 7 or 8

Parity: Even, Odd, or None

Stop bits: 1 or 2

Flow control: XON/XOFF or hardware

Ethernet (modern): - FTP, SFTP, or manufacturer protocol - Faster, more reliable - Supports large file transfers

USB (some controls): - Direct USB drive insertion - USB cable to control

Drip-Feed Programs

For programs larger than control memory:

(DNC software sends program line-by-line as machine executes)

1. Connect computer to machine via serial/Ethernet
2. Load program in DNC software
3. Start drip-feed mode
4. Machine requests blocks as needed
5. Computer sends next block when requested

DNC software: - Predator DNC - CIMCO DNC-Max - OpenDNC - Machine vendor software (Haas DNC, etc.)

File Transfer Best Practices

1. Verify file integrity:

(After transfer, compare file sizes)

Original: 12,345 bytes

Transferred: 12,345 bytes [check]

(Check first and last lines of program)

Original: % ... M30 %

Transferred: % ... M30 % [check]

2. Use correct line endings: - Windows: CR+LF (CRLF) - Linux/Mac: LF (LF) - Most CNC controls accept either

3. Character encoding: - ASCII only (no Unicode, special characters) - Avoid extended characters in comments

4. File naming: - No spaces (use underscores or hyphens) - Short names (8.3 format for older controls) - Example: PART_001.NC, BRK100-OPN10.NC

Program Validation Checklist

Before running any new program:

- ☐ Program simulated in CAMotics/NC Viewer
- ☐ Material removal simulation completed (if available)
- ☐ No collisions detected with fixtures/clamps
- ☐ All tool numbers present in tool library
- ☐ Tool lengths measured and offsets set
- ☐ Work offsets set and verified (touch-off)
- ☐ Feed rates appropriate for material and tools
- ☐ Spindle speeds within tool and machine limits
- ☐ Rapids clear all obstacles by safe margin
- ☐ Dry run completed with Z-axis raised
- ☐ First article inspection planned
- ☐ Safety equipment in place (guards, e-stop accessible)

Common Simulation Findings

Feed Rate Errors

Missing F-word:

G01 X10 Y10 (No feed rate - machine stalls)

Simulation shows: Tool freezes at start of move, alarm condition.

Fix:

G01 X10 Y10 F500

Work Offset Errors

Wrong offset selected:

G55 (Should be G54)

G00 X0 Y0 (Tool goes to wrong position)

Simulation shows: Tool approaches unexpected location, possible collision.

Fix:

G54 (Correct work offset)
G00 X0 Y0

Tool Length Not Applied

Missing G43:

T01 M06
(G43 H01 MISSING)
G00 Z0 (Tool plunges into table)

Simulation shows: Tool crashes into stock or table.

Fix:

T01 M06
G43 H01 (Apply tool length offset)
G00 Z0

Clearance Errors

Rapids at cutting depth:

G01 Z-10 F100 (Cut to depth)
G00 X100 Y50 (Rapid at Z-10, still in part)

Simulation shows: Tool drags through part during rapid.

Fix:

G01 Z-10 F100
G00 Z5.0 (Retract first)
G00 X100 Y50 (Safe rapid)

Advanced Verification Techniques

Chip Load Validation

Calculate and verify chip loads match tooling recommendations:

$$\text{Chip Load} = \frac{F}{S \times N \times Z}$$

Where: - F = Feed rate (mm/min) - S = Spindle speed (RPM) - N = Number of teeth - Z = Number of flutes

Example:

S2000 M03 (2000 RPM)
G01 X100 F800 (800 mm/min)
(Tool: 4-flute end mill)

Chip load = 800 / (2000 x 1 x 4) = 0.1 mm/tooth [check]

Cutting Force Simulation

Advanced simulators estimate cutting forces: - Detect overload conditions - Predict tool deflection - Optimize feeds and speeds - Identify chatter risk

Software: VERICUT Force, HSMWorks Force Module

Cycle Time Analysis

Breakdown by operation:

Total cycle time: 12:35

Rapids: 1:20 (10.5%)

Roughing: 7:45 (61.7%)

Finishing: 2:15 (17.9%)

Tool changes: 1:15 (9.9%)

Optimization targets: - Reduce air cutting (rapids) - Increase feed rates where safe - Combine operations to reduce tool changes

Key Takeaways

1. **Never run unverified programs** on physical machines
2. **Three-stage verification:** Toolpath check -> Material removal -> Dry run
3. **Free tools** (CAMotics, NC Viewer) sufficient for basic verification
4. **Commercial simulators** (VERICUT) essential for complex/high-value work
5. **Dry run procedures** catch setup and offset errors before cutting
6. **DNC communication** enables large file transfer and drip-feeding
7. **Common errors** detected: Missing feed rates, wrong offsets, clearance issues
8. **First article inspection** validates program produces correct part
9. **Simulation workflow** is systematic: Quick check -> Detailed sim -> Dry run -> First part
10. **Professional shops** use multi-stage verification for all new programs

Next: Section 15.12 - Conclusion

Previous: Section 15.10 - Control System Dialects

Section 15.12 - Conclusion and Integration

Module Summary

This module has provided comprehensive coverage of G-code programming from foundational syntax through advanced parametric techniques. You've learned the language that directly controls CNC machines, bridging the gap between design intent and physical reality.

Key Concepts Covered

Section 15.1 - Introduction: - G-code's role as the universal CNC language - Historical context and standards (ISO 6983) - Module scope and learning objectives

Section 15.2 - G-Code Structure: - Block format and word structure - Address codes (G, M, X, Y, Z, F, S, T) - Modal vs. non-modal commands - Program organization and syntax conventions

Section 15.3 - Motion Commands: - G00 (rapid positioning), G01 (linear interpolation) - G02/G03 (circular interpolation) - Feed rate control and trajectory planning - Helical interpolation and path blending

Section 15.4 - Coordinate Systems: - Machine vs. work coordinate systems - Work offsets (G54-G59) - Absolute (G90) and incremental (G91) positioning - Tool length and cutter radius compensation

Section 15.5 - Auxiliary Functions: - M-codes for spindle control (M03/M04/M05) - Coolant control (M08/M09) - Tool changes (M06) and program flow (M00/M01/M30) - Subprogram calls (M98/M99)

Section 15.6 - Canned Cycles: - Drilling cycles (G81-G83) - Tapping cycles (G84) - Boring cycles (G85-G89) - Return modes (G98/G99) and parameter usage

Section 15.7 - Programming Best Practices: - Program structure and documentation - Safety initialization and error prevention - Toolpath optimization - Version control and maintainability

Section 15.8 - Post-Processing: - CAM to G-code translation - Post-processor architecture and configuration - Customization techniques - Control-specific output formatting

Section 15.9 - Advanced Features: - Variables and parametric programming - Mathematical expressions and trigonometry - Conditional logic and loops - Practical applications (bolt circles, adaptive feeds)

Section 15.10 - Control Dialects: - FANUC, Siemens, Heidenhain, Haas, LinuxCNC differences - Syntax variations and feature availability - Adaptation strategies between controls

Section 15.11 - Simulation and Verification: - Toolpath visualization and material removal simulation - Dry run procedures and first article inspection - DNC communication - Program validation workflows

Integration with Previous Modules

G-code programming builds upon and integrates with every previous module in this course:

Mechanical Systems (Modules 1-3)

Module 1-2: Frame and Vertical Axis - G-code coordinates map to physical X, Y, Z, A, B, C axes - Understanding machine kinematics essential for safe programming - Travel limits in G-code must respect mechanical constraints

Module 3: Linear Motion Systems - Feed rates (F-word) determined by ball screw pitch and motor limits - Positioning accuracy affects achievable tolerances in programs - Rapid rates (G00) limited by mechanical acceleration capabilities

Connection:

G00 X500 Y400	(Must be within travel limits)
G01 X100 F500	(Feed rate respects ball screw/servo limits)

Control Electronics (Module 4)

Stepper vs. Servo Systems: - Step/direction signals generated from G01 feed rates - Encoder feedback enables closed-loop position verification - Servo systems allow higher feed rates and accelerations

Real-time execution: - G-code interpreter runs in real-time control loop - Look-ahead buffer optimizes trajectory planning - Step generation timing critical for smooth motion

Connection:

```
G64 P0.01          (Path blending requires look-ahead buffer)
G01 X50 F2000      (Controller translates to step pulses)
```

Process-Specific Modules (Modules 5-13)

Module 5: Plasma Cutting - Pierce delays, kerf compensation in G-code - THC (Torch Height Control) integrated via M-codes - 2D profiling with G01, G02, G03

Module 6: Spindle Systems - S-word (spindle speed) commands VFD or servo spindle - M03/M04/M05 control spindle via relay or ModBus - Dwell times (G04) for spindle acceleration

Module 7: Fiber Laser - Laser power control via analog output or M-codes - Focus control integrated into Z-axis motion - Cut speed optimization in feed rates

Module 8: Waterjet - Abrasive feed rate tied to cutting feed (F-word) - Pierce delays before cutting moves - Multi-pass depth control via Z-increments

Module 9-10: Robotics - 6-axis robot arms programmed with extended G-code - A, B, C rotary axes for tool orientation - Coordinated motion of all axes

Module 11-12: Large Format and Hybrid - G-code scales to large travel ranges - Multi-process machines use M-codes to switch tools/processes - Parametric programming for part families

Module 13: EMI/EMC - Proper grounding prevents G-code communication errors - Shielded cables for encoder and step signals - Noise immunity essential for reliable motion

Module 14: LinuxCNC and HAL - G-code interpreter runs in LinuxCNC task controller - HAL pins connect G-code M-codes to physical I/O - Real-time kernel ensures deterministic G-code execution

Critical connection:

```
# LinuxCNC HAL configuration
net spindle-on motion.spindle-on => spindle.enable
net spindle-speed motion.spindle-speed-out => spindle.speed-in

# G-code "M03 S2000" triggers these HAL connections
```

Integration with Module 16 (CAD/CAM/DFM)

Module 16 Preview: - CAD models define part geometry - CAM generates toolpaths from CAD - Post-processors (covered here) convert toolpaths to G-code - DFM principles guide machinable design

Workflow:

```
CAD Design (Module 16)
  down
CAM Toolpath Calculation (Module 16)
  down
Post-Processor (Module 15)
  down
G-Code Program (Module 15)
  down
```

CNC Controller (Module 14)
down
Motion Control (Modules 1-4)
down
Process Execution (Modules 5-13)
down
Physical Part

Practical Application Path

Level 1: Basic Operator

Skills from this module: - Read and understand G-code programs - Verify programs with simulation - Load programs via DNC - Set work offsets and tool offsets - Run dry runs and first articles

Typical tasks: - Load CAM-generated programs - Perform setup and touch-off - Monitor program execution - Troubleshoot basic errors

Level 2: CNC Programmer

Skills from this module: - Write simple programs by hand - Edit CAM output for optimization - Configure post-processors - Use canned cycles effectively - Apply best practices for safety and efficiency

Typical tasks: - Create programs for simple 2D parts - Modify CAM programs for different setups - Customize post-processors for shop machines - Debug and fix program errors

Level 3: Advanced Programmer/Engineer

Skills from this module: - Parametric programming with variables - Macro programming for automation - Custom post-processor development - Multi-axis and complex surface programming - Process optimization through G-code

Typical tasks: - Create parametric part families - Develop custom cycles and macros - Integrate probing and adaptive control - Optimize programs for maximum efficiency - Train others in G-code programming

Real-World Applications

Job Shop Programming

Scenario: Small batch production, many different parts

G-code skills applied: - Quick program creation for simple parts - CAM with post-processing for complex parts - Efficient use of canned cycles - Subprograms for common features (bolt circles, pockets) - Rapid setup with proper documentation

Production Machining

Scenario: High-volume, few part types

G-code skills applied: - Optimization for minimum cycle time - Proven programs archived and versioned - Parametric programming for part families - Integration with tool life management - DNC for reliable program transfer

Prototype and R&D

Scenario: One-off parts, frequent changes

G-code skills applied: - Hand coding for quick iterations - CAM for complex geometry - Simulation to prevent costly errors - Adaptive programming with sensors/probing - Experimental feeds and speeds

Custom Machine Building

Scenario: Non-standard machines, unique processes

G-code skills applied: - Custom M-codes for special operations - Post-processor development from scratch
- Integration with LinuxCNC HAL - Kinematics for non-Cartesian machines - Process-specific programming techniques

Continuing Education

Recommended Next Steps

- 1. Hands-on practice:** - Simulator: Install CAMotics or LinuxCNC simulator - Write simple programs: Rectangles, bolt circles, pockets - Edit CAM output: Modify post-processed programs - Run programs: On actual machine (with supervision) or simulator
- 2. CAM software proficiency:** - Learn Fusion 360 (free for hobbyists) - Or Mastercam, SolidCAM, HSMWorks (professional) - Practice toolpath generation - Configure post-processors for your machines
- 3. Advanced topics:** - 5-axis programming - High-speed machining (HSM) techniques - Adaptive machining with force feedback - In-process measurement and probing - Custom macro development
- 4. Control-specific training:** - FANUC operator/programmer courses - Siemens Sinumerik training - Heidenhain TNC programming - LinuxCNC integrator training

Resources for Further Learning

Books: - “CNC Programming Handbook” by Peter Smid - “Fanuc CNC Custom Macros” by Peter Smid - “G-Code Programming” by John Wilding - “Machinery’s Handbook” (reference for speeds/feeds/materials)

Online courses: - Titan FANUC Training (YouTube) - CNC Cookbook G-Wizard training - Fusion 360 CAM tutorials (Autodesk) - LinuxCNC documentation and forum

Communities: - Practical Machinist forum - CNCZone forums - LinuxCNC forum - Reddit r/CNC, r/Machinists

Software for practice: - CAMotics (free simulation) - NC Viewer (web-based visualization) - LinuxCNC (free, open-source control) - Fusion 360 (free for hobbyists, includes CAM)

Safety Reminders

G-code programming carries significant responsibility. Always remember:

- 1. Simulation is mandatory** - Never run unverified programs - Simulate every new program, every setup change - Dry run above work surface before cutting
- 2. Verify all offsets** - Work offsets (G54-G59) - Tool length offsets (G43 H___) - Tool radius offsets (G41/G42 D___)
- 3. Check feed rates and speeds** - F-word appropriate for tool and material - S-word within tool and machine limits - No zero feed rates (causes stalls)
- 4. Proper initialization** - G21/G20 (units), G90/G91 (distance mode) - G40 G49 G80 (cancel all compensations) - G54 (work offset selection)
- 5. Emergency preparedness** - Know e-stop location - Understand feed override controls - Single-block mode for critical operations - Operator trained and attentive

The programmer is responsible for safe programs. Machine capability includes significant destructive force. Program with care.

Final Thoughts

G-code is simultaneously simple and powerful—a text-based language that precisely controls sophisticated machines. Mastery of G-code programming provides several key advantages:

- 1. Machine independence** - Understand any CNC machine's fundamental operation - Adapt to new controls quickly - Troubleshoot issues across different platforms
- 2. Control and optimization** - Hand-edit CAM output for better results - Create custom operations CAM can't generate - Optimize cycle times beyond automated solutions
- 3. Career versatility** - Job shop programmer - Production engineer - CAM specialist - Custom machine builder - CNC trainer/educator
- 4. Problem-solving capability** - Debug failed programs - Recover from crashes and interruptions - Adapt to unique challenges - Innovate new techniques
- 5. Foundation for advanced topics** - Multi-axis programming - Adaptive machining - In-process measurement - Lights-out manufacturing

This module has equipped you with the knowledge to write, read, debug, and optimize G-code programs. Combined with the mechanical, electronic, and control system understanding from previous modules, you now possess a complete foundation for CNC engineering.

Course Completion Path

You've now covered: - [check] Modules 1-3: Mechanical systems - [check] Module 4: Control electronics - [check] Modules 5-13: Process-specific systems - [check] Module 14: LinuxCNC and HAL - [check] Module 15: G-Code programming (this module)

Next: - Module 16: CAD, CAM, and Design for Manufacturability

After Module 16, you will have complete CNC engineering competency: - Design machines (Modules 1-3) - Wire and control them (Module 4, 14) - Integrate specialized processes (Modules 5-13) - Program them (Module 15) - Design manufacturable parts (Module 16)

Acknowledgments

G-code programming knowledge stands on decades of CNC development by engineers, machinists, and programmers worldwide. The standards (ISO 6983, EIA RS-274) emerged from collaborative industrial efforts. Open-source projects like LinuxCNC democratize access to CNC control technology.

Special recognition: - MIT Servomechanisms Lab (NC origin) - ISO Technical Committee (standardization) - FANUC, Siemens, Heidenhain (control development) - LinuxCNC developers and community - CAM software developers - Machinists and programmers who share knowledge freely

Conclusion

G-code programming is a craft that combines technical precision with practical problem-solving. Like any craft, mastery comes through study and practice. This module has provided the foundation—now it's your turn to write programs, make chips, and build your expertise.

Welcome to the world of CNC programming. The machines await your commands.

Previous: Section 15.11 - Simulation and Verification

Next Module: Module 16 - CAD, CAM, and Design for Manufacturability

Return to: Module 15 Overview

Section 15.2 - G-Code Structure and Syntax

Overview

G-code programs are composed of sequential blocks of ASCII text, each specifying machine actions through address codes and numerical parameters. Understanding the fundamental structure of G-code blocks, the meaning of address letters, and the distinction between modal and non-modal commands is essential for reading, writing, and debugging CNC programs.

This section provides a comprehensive examination of G-code syntax, block organization, command types, and formatting conventions used across modern CNC control systems.

Block Structure

Anatomy of a G-Code Block

A G-code block is a single line of code terminated by a line feed (LF) or carriage return/line feed (CR/LF). Each block contains one or more words that specify machine operations.

Basic block format:

```
N100 G01 X10.5 Y20.3 Z-5.0 F500 S2000 M03
```

Block components:

Component	Example	Description
Line number	N100	Optional sequence identifier
Preparatory function	G01	Motion or mode command
Coordinate data	X10.5 Y20.3 Z-5.0	Axis endpoint positions
Feed rate	F500	Cutting speed (units/min)
Spindle speed	S2000	Rotation speed (RPM)
Miscellaneous function	M03	Auxiliary command (spindle on CW)

Word Format

A **word** consists of an address letter followed by a numerical value:

```
Letter + Number = Word
  G  +   01   = G01
  X  +  10.5  = X10.5
  F  +   500  = F500
```

Word syntax rules:

1. **Address letter:** Single uppercase character (A-Z)
2. **Numerical value:** Integer or decimal, signed or unsigned
3. **No spaces:** Between letter and number (most controls)
4. **Leading zeros:** Optional in most dialects (G01 = G1)
5. **Trailing zeros:** Required for precision (X10.0 not X10.)
6. **Sign convention:** + assumed if not specified, - must be explicit

Block Execution Order

Words within a block are not executed left-to-right. The controller processes them in a fixed sequence regardless of their order in the block:

Standard execution order:

1. **Comments** - Processed/ignored first

2. **N (Sequence number)** - Stored for reference
3. **G (Preparatory)** - Sets motion mode
4. **X, Y, Z, A, B, C** - Axis coordinates
5. **F (Feed rate)** - Sets feed for this and subsequent moves
6. **S (Spindle speed)** - Sets RPM for this and subsequent operations
7. **T (Tool)** - Selects tool (prepared for change)
8. **M (Miscellaneous)** - Auxiliary functions last

Example demonstrating order independence:

```
X10 G01 Y20 F100 Z5    (Same result as...)
G01 X10 Y20 Z5 F100    (... written in logical order)
```

Both blocks produce identical motion: linear interpolation to X10 Y20 Z5 at 100 units/min feed rate.

Address Codes

Standard Address Letters

ISO 6983 defines specific meanings for each letter address:

Letter	Function	Example	Units/Range
A	Rotary axis (around X)	A45.0	Degrees
B	Rotary axis (around Y)	B90.0	Degrees
C	Rotary axis (around Z)	C180.0	Degrees
D	Tool radius offset number	D01	Index
E	Reserved (not standardized)	-	Varies
F	Feed rate	F500	mm/min or in/min
G	Preparatory function	G01	Code number
H	Tool length offset number	H03	Index
I	Arc center X offset	I5.0	Distance
J	Arc center Y offset	J-3.0	Distance
K	Arc center Z offset	K2.0	Distance
L	Loop count (subprograms)	L10	Iterations
M	Miscellaneous function	M08	Code number
N	Sequence number	N100	Index
O	Program number	O1234	Index
P	Dwell time / parameter	P500	Milliseconds
Q	Peck increment (drilling)	Q2.0	Distance
R	Arc radius / retract plane	R5.0 or R10.0	Distance
S	Spindle speed	S2000	RPM
T	Tool selection	T05	Tool number
U	Secondary X-axis (parallel)	U10.0	Distance
V	Secondary Y-axis (parallel)	V5.0	Distance
W	Secondary Z-axis (parallel)	W-2.0	Distance
X	Primary X-axis	X100.0	Distance
Y	Primary Y-axis	Y50.0	Distance
Z	Primary Z-axis	Z-10.0	Distance

Extended Addressing

Some controls use additional conventions:

- **# variables:** #100 = 5.5 (parametric programming)
- **E-axis:** Extruder position (3D printing)
- **Expressions:** #1 = [#2 + #3 * COS[45]]

Modal and Non-Modal Commands

Modal Commands (Sticky)

Modal commands remain active until explicitly changed by another command in the same group. They define the machine's current state.

Example of modal behavior:

```
G01 F100      (Linear mode active, feed = 100)
X10 Y10       (Still G01 at F100)
X20 Y20       (Still G01 at F100)
G00           (Now rapid mode active)
X30 Y30       (Rapid move, no feed rate)
```

Once G01 is specified, all subsequent coordinate moves use linear interpolation until a different motion mode (G00, G02, etc.) is commanded.

Non-Modal Commands (Single Block)

Non-modal commands execute only in the block where they appear and do not affect subsequent blocks.

Examples of non-modal commands: - **G04** (Dwell) - Pause for specified time in current block only - **G28** (Return to home) - Execute homing in current block only - **G92** (Set work coordinate) - Set offset in current block only

Example:

```
G04 P1000     (Dwell 1 second, then continue)
X10 Y10       (Move continues in previous modal state)
```

Modal Groups

Commands are organized into modal groups. Only one command from each group can be active at a time.

Common modal groups:

Group	Function	Commands
Motion	Move type	G00, G01, G02, G03, G04, G80-G89
Plane selection	Arc plane	G17 (XY), G18 (XZ), G19 (YZ)
Distance mode	Coordinate type	G90 (absolute), G91 (incremental)
Units	Measurement system	G20 (inches), G21 (millimeters)
Cutter compensation	Tool radius	G40 (off), G41 (left), G42 (right)
Tool length	Z offset	G43 (on), G49 (off)
Coordinate system	Work offset	G54, G55, G56, G57, G58, G59
Path control	Trajectory mode	G61 (exact stop), G64 (continuous)
Return mode	Canned cycle return	G98 (initial Z), G99 (R-plane)

Group conflict example (ERROR):

```
G01 G00 X10 Y10    (INVALID: Both G01 and G00 in motion group)
```

The controller will reject this block or execute only the last command (G00).

Active Modal State

The controller maintains the active state of each modal group. This state persists: - Through program execution - After cycle stops (on most controls) - Across program restarts (unless reset)

Best practice: Always initialize modal states at program start to ensure predictable behavior.

Program Structure

Typical Program Layout

A complete G-code program follows this general structure:

```
%                (Program start flag - optional)
01234            (Program number)
(PART: SAMPLE BRACKET) (Comment: part identification)
(MATERIAL: 6061-T6 AL) (Comment: material specification)
(SETUP: VISE JAW 1)   (Comment: fixturing information)

(--- INITIALIZATION ---)
G21 G90 G17        (Metric, absolute, XY plane)
G54                (Work coordinate system 1)
G49                (Cancel tool length offset)
G40                (Cancel cutter radius compensation)

(--- TOOL 1: 12MM END MILL ---)
T01 M06            (Select and change to tool 1)
G43 H01            (Apply tool length offset 1)
S2000 M03          (Spindle 2000 RPM clockwise)
G00 X0 Y0          (Rapid to start position)
G00 Z5.0           (Rapid to safe Z)
M08                (Coolant on)

(--- MACHINING OPERATIONS ---)
G01 Z-5.0 F100     (Plunge into material)
G01 X50.0 Y0 F500  (Cut to endpoint)
G01 Z5.0 F100      (Retract)

(--- CLEANUP ---)
G00 Z50.0          (Rapid to safe Z)
M09                (Coolant off)
M05                (Spindle off)
G28 G91 Z0         (Home Z-axis)
G28 X0 Y0          (Home XY axes)
G90                (Restore absolute mode)
M30                (Program end, rewind)
%                (Program end flag - optional)
```

Header Block

The program header establishes context and initial conditions:

```
01234 (PROGRAM NUMBER AND NAME)
(PROGRAMMER: SMITH, J.)
(DATE: 2025-01-15)
(PART: BRK-100-REV-C)
(MATERIAL: 6061-T6 ALUMINUM)
(STOCK: 6.25 X 4.25 X 1.00)
(ORIGIN: CENTER-CENTER-TOP)
```

Safety Block

The safety/initialization block sets known modal states:

G21	(Metric units)
G90	(Absolute positioning)
G17	(XY plane selection)
G40	(Cutter compensation off)
G49	(Tool length offset off)
G80	(Cancel canned cycles)
G54	(Work offset 1)
G94	(Feed per minute mode)

Tool Change Block

Tool changes follow a standard sequence:

```
(--- TOOL 2: 6MM DRILL ---)
M05          (Stop spindle)
M09          (Coolant off)
G00 Z50.0    (Retract to safe Z)
T02 M06      (Change to tool 2)
G43 H02      (Tool length offset 2)
S3000 M03    (Spindle 3000 RPM)
M08          (Coolant on)
```

Program Termination

Programs end with cleanup and termination codes:

```
M05          (Spindle off)
M09          (Coolant off)
G28 G91 Z0   (Home Z)
G28 X0 Y0    (Home XY)
G90          (Restore absolute)
M30          (Program end and rewind)
```

Syntax Conventions

Line Numbers (N-words)

Line numbers provide reference points for program editing and troubleshooting:

```
N10 G21 G90 G54
N20 T01 M06
N30 G43 H01
N40 S2000 M03
```

Conventions: - Increment by 5 or 10 to allow insertions - Not required for execution (controller ignores) - Useful for editing, debugging, restart points - Some CAM systems omit to reduce file size

Comments

Comments document program intent and are ignored during execution:

Parenthesis comments:

```
G01 X10 Y20 (MOVE TO POSITION 1)
```

Semicolon comments (LinuxCNC, some controls):

```
G01 X10 Y20 ; Move to position 1
```

Best practices: - Explain operations, not obvious syntax - Document tool descriptions, cutting parameters
- Note critical setup requirements - Include revision history for program changes

Case Sensitivity

Most CNC controls are **case-insensitive**:

G01 X10 Y20 (Same as...)
g01 x10 y20 (... lowercase)

Convention: Use uppercase for consistency and readability.

Whitespace

Spaces between words (modern controls):

G01 X10 Y20 F100 (Readable format)

No spaces (legacy/compact format):

G01X10Y20F100 (Compact format)

Most modern controls accept either format, but spaced format is preferred for human readability.

Decimal and Precision

Decimal Point Format

Always use decimal points for fractional values:

X10.5 (Correct: 10.5 units)
X10,5 (WRONG: Comma not recognized)
X10. (WRONG: Trailing decimal ambiguous)

Leading Zeros

Leading zeros are optional on most controls:

G01 (Same as...)
G1 (... no leading zero)

X005.0 (Same as...)
X5.0 (... no leading zeros)

Convention: Use leading zeros for consistency (G01, G00, M03).

Trailing Zeros and Precision

Specify precision appropriate to machine capability:

X10.0 (0.1 resolution - standard milling)
X10.00 (0.01 resolution - precision work)
X10.000 (0.001 resolution - grinding, EDM)
X10.0000 (0.0001 resolution - ultra-precision)

Best practice: Match resolution to machine capability and part tolerance.

Special Characters

Program Delimiters

- % - Program start/end marker (DNC/RS-232 transmission)

- **EOB** - End of block (line feed character, invisible)

Reserved Characters

Characters with special meaning:

Character	Function
()	Comment delimiters
%	Program boundary
;	Comment (some controls)
#	Variable prefix
[]	Expression delimiters
/	Block skip (optional execution)

Block Skip

The / character at the start of a block makes it conditionally executable:

/G01 X10 Y20 (Executed only if block skip switch is OFF)

Used for: - Optional roughing passes - Debug output - Conditional probing operations

Error Prevention

Common Syntax Errors

Missing feed rate:

G01 X10 Y10 (ERROR: Feed rate not defined)

Solution: Always specify F-word before or with first G01:

G01 X10 Y10 F500 (Correct)

Ambiguous arc parameters:

G02 X10 Y10 (ERROR: Missing I, J, or R)

Solution: Specify arc center or radius:

G02 X10 Y10 I5 J0 (Correct: center offset)

G02 X10 Y10 R5 (Correct: radius)

Modal group conflicts:

G01 G00 X10 (ERROR: Both linear and rapid)

Solution: Use one motion command:

G01 X10 (Correct)

Syntax Validation

Before running a program:

1. **Use a syntax checker** - CAM software, LinuxCNC verify mode
2. **Simulation** - Visualize toolpath for unexpected moves
3. **Dry run** - Execute with feed override at 0% or in simulation mode
4. **Single block mode** - Step through line-by-line for new programs

Key Takeaways

1. **G-code blocks** consist of words (letter + number) executed in a defined order
2. **Address letters** (G, M, X, Y, Z, F, S, etc.) specify different machine functions
3. **Modal commands** remain active until changed; **non-modal** execute once
4. **Modal groups** organize commands; only one per group can be active
5. **Program structure** includes initialization, tool changes, operations, and cleanup
6. **Comments** document intent; **line numbers** aid debugging
7. **Syntax rules** include decimal format, precision, and special character usage
8. **Error prevention** requires understanding modal state and parameter requirements

Next: Section 15.3 - Motion Commands

Previous: Section 15.1 - Introduction

Section 15.3 - Motion Commands

Overview

Motion commands are the foundation of CNC programming, translating geometric paths into coordinated axis movements. The four primary motion modes—rapid positioning (G00), linear interpolation (G01), and circular interpolation (G02/G03)—enable the creation of virtually any 2D or 3D toolpath.

This section provides comprehensive coverage of motion command syntax, trajectory planning, feed rate control, and the mathematical principles underlying linear and circular interpolation.

G00 - Rapid Positioning

Function

G00 commands the machine to move all specified axes simultaneously to the endpoint at maximum traverse speed. This is a **non-cutting move** used for positioning between operations.

Syntax:

```
G00 X__ Y__ Z__ A__ B__ C__
```

Example:

```
G00 X50.0 Y25.0 Z10.0    (Rapid to position, Z safe above work)
```

Characteristics

Multi-axis coordination: - All axes move simultaneously - Each axis travels at its maximum speed - Motion follows a straight line in machine coordinates - **Not guaranteed** to follow a straight line in work coordinates when rotary axes are involved

Speed: - Rapid rate set by machine parameters (typically 500-2000 IPM / 12000-50000 mm/min) - Independent of F-word (feed rate) - Limited by machine acceleration and servo performance

Safety considerations: - **Never use G00 with tool in contact** with workpiece - Always retract Z-axis before rapid XY moves - Check clearances for fixtures, clamps, part edges - Use safe Z heights appropriate to setup

Trajectory Planning

The path taken during G00 depends on the controller:

Type 1: Simultaneous arrival (ideal line) - All axes reach endpoint at the same time - Follows a straight line in space - Most common on modern CNC controls

Type 2: Independent axis (dog-leg) - Each axis completes at different times - Fastest axis arrives first, path is not straight - Common on older controls or when axes have very different speeds

Example comparison:

```
G00 X100 Y100    (From origin)
```

Simultaneous arrival: Diagonal line from (0,0) to (100,100) **Independent axis:** L-shaped path if X-axis faster than Y-axis

Best practice: Never rely on G00 path shape. Always position with adequate clearance.

Safe Retract Strategy

Standard practice for safe rapids:

```

G01 Z-10.0 F100      (Cutting at depth)
G00 Z5.0             (Retract Z to safe height)
G00 X100.0 Y50.0     (Rapid XY to next position)
G00 Z-8.0            (Rapid down to near-depth)
G01 Z-10.0 F100      (Feed to cutting depth)

```

This sequence ensures: 1. Z-axis clears workpiece before XY rapid 2. No collision with fixtures or clamps 3. Controlled feed for final approach

G01 - Linear Interpolation

Function

G01 commands coordinated linear motion of all specified axes at a controlled feed rate. This is the primary **cutting move** for straight-line toolpaths.

Syntax:

```
G01 X__ Y__ Z__ A__ B__ C__ F__
```

Example:

```
G01 X50.0 Y25.0 Z-5.0 F500    (Linear cut to endpoint at 500 units/min)
```

Characteristics

Coordinated motion: - All axes move in synchronization - Motion follows a straight line in Cartesian space - Feed rate applies to the resultant velocity vector - Arrival at endpoint is simultaneous for all axes

Feed rate: - Specified by F-word in units per minute (G94 mode) - Remains modal until changed - Must be set before or with first G01 command - Applies to subsequent G01 moves until changed

Feed Rate Calculation

The feed rate F specifies the velocity of the **tool tip** along the programmed path, not individual axis speeds.

For linear motion, resultant feed rate:

$$F_{resultant} = \sqrt{(v_X)^2 + (v_Y)^2 + (v_Z)^2}$$

Where v_X , v_Y , v_Z are individual axis velocities.

Example:

```
G01 X10.0 Y0 F100      (Move 10mm in X at 100 mm/min)
```

- Time = Distance / Feed = 10 / 100 = 0.1 minutes = 6 seconds
- $v_X = 100$ mm/min, $v_Y = 0$

```
G01 X10.0 Y10.0 F100    (Move diagonally at 100 mm/min resultant)
```

- Path length = $\sqrt{10^2 + 10^2} = 14.14$ mm
- Time = 14.14 / 100 = 0.1414 minutes = 8.49 seconds
- $v_X = 70.71$ mm/min, $v_Y = 70.71$ mm/min
- Resultant = $\sqrt{70.71^2 + 70.71^2} = 100$ mm/min

The controller automatically calculates individual axis speeds to maintain the programmed feed rate along the path.

Multi-Axis Linear Motion

G01 supports coordinated motion of any combination of axes:

XY plane cutting:

G01 X50.0 Y25.0 F500 (2-axis move)

3-axis contouring:

G01 X50.0 Y25.0 Z-5.0 F500 (3-axis move)

4-axis indexed:

G01 X50.0 A90.0 F300 (Linear + rotary)

5-axis simultaneous:

G01 X50.0 Y25.0 Z-5.0 A45.0 B30.0 F200 (All 5 axes)

Incremental vs. Absolute

G01 respects the current distance mode (G90/G91):

Absolute mode (G90):

G90 (Absolute mode)
G01 X10.0 Y10.0 F100 (Move to X=10, Y=10)
G01 X20.0 Y20.0 (Move to X=20, Y=20)

Incremental mode (G91):

G91 (Incremental mode)
G01 X10.0 Y10.0 F100 (Move +10 in X, +10 in Y from current)
G01 X10.0 Y10.0 (Move another +10 in X, +10 in Y)

Mixed mode (some controls):

G90 G91.1 (Absolute distance, incremental arc centers)
G01 X20.0 Y20.0 (Absolute coordinate)

G02 and G03 - Circular Interpolation

Function

G02 (clockwise) and **G03** (counterclockwise) command circular or helical motion at controlled feed rate. Essential for arcs, radii, and circular pockets.

Syntax (center format):

G02 X__ Y__ I__ J__ F__ (CW arc with center offset)
G03 X__ Y__ I__ J__ F__ (CCW arc with center offset)

Syntax (radius format):

G02 X__ Y__ R__ F__ (CW arc with radius)
G03 X__ Y__ R__ F__ (CCW arc with radius)

Direction Convention

The direction (CW vs. CCW) is defined by viewing the plane from the **positive axis** perpendicular to the plane:

G17 (XY plane): View from +Z looking down - G02 = clockwise - G03 = counterclockwise

G18 (XZ plane): View from +Y looking left **G19 (YZ plane):** View from +X looking right

Center Format (I, J, K)

The arc center is specified as an **offset from the start point**, not an absolute coordinate.

Parameters: - **I:** Offset from start X to arc center X - **J:** Offset from start Y to arc center Y - **K:** Offset from start Z to arc center Z (helical)

Example:

```
G90 G17                (Absolute mode, XY plane)
G00 X0 Y0              (Start position)
G01 X10.0 Y0 F500      (Move to arc start)
G02 X10.0 Y20.0 I0 J10.0 (CW arc, center at X10 Y10)
```

Calculation: - Start point: (10, 0) - End point: (10, 20) - Center offset: I=0, J=10 - Arc center: (10+0, 0+10) = (10, 10) - Radius: 10 units - Arc: 180 deg semicircle from bottom to top

Visual:

```
      End (10,20)
      |
-----C----- Center (10,10)
      |
      Start (10,0)
```

Radius Format (R)

Simpler syntax using radius directly:

```
G02 X10.0 Y20.0 R10.0 F500    (CW arc with radius 10)
```

Advantages: - Easier to read and understand - No offset calculation required - Common for simple arcs

Limitations: - Ambiguous for arcs > 180 deg (two possible arcs) - Cannot describe a full circle (start = end)

Radius sign convention: - **R positive:** Arc <= 180 deg (minor arc) - **R negative:** Arc > 180 deg (major arc)

Example:

```
G00 X0 Y0
G02 X10.0 Y10.0 R10.0    (90 deg arc, minor)
```

vs.

```
G00 X0 Y0
G02 X10.0 Y10.0 R-10.0   (270 deg arc, major)
```

Arc Validation

The controller validates arc geometry before execution. Common errors:

Radius mismatch:

```
G00 X0 Y0
G02 X20.0 Y0 I5.0 J0      (ERROR: Start radius != end radius)
```

- Start radius: $\sqrt{5^2 + 0^2} = 5$
- End radius: $\sqrt{(5-20)^2 + 0^2} = 15$
- **Mismatch:** Controller rejects or alarms

Tolerance: Most controls allow small discrepancies (0.001-0.01mm) due to rounding.

Helical Interpolation

Adding a Z-component creates a helical path:

```
G17                                (XY plane for arc)
G00 X10.0 Y0 Z0
G02 X10.0 Y0 Z-10.0 I-10.0 J0 F500    (Helical CW, full circle descending 10mm)
```

Applications: - Thread milling - Circular pockets with depth - Spiral ramps into material

Feed rate applies to the 3D helix length:

$$L_{helix} = \sqrt{L_{arc}^2 + \Delta Z^2}$$

Full Circle

A full 360 deg circle requires **I, J, K** format (not R):

```
G00 X10.0 Y0
G02 X10.0 Y0 I-10.0 J0 F500    (Full circle, center at origin)
```

- Start point: (10, 0)
- End point: (10, 0) - same as start
- Center offset: I=-10, J=0
- Arc center: (10-10, 0+0) = (0, 0)
- Radius: 10 units

Arc Feed Rate

Feed rate applies to the arc **circumference**, not radius:

```
G02 X10.0 Y10.0 I5.0 J0 F500
```

The tool moves along the arc path at 500 units/min, automatically adjusting individual axis speeds for circular motion.

Calculation example:

Arc length = $r \times \theta$ (radians)

For 90 deg arc with radius 10mm: - Arc length = $10 \times (\pi/2) = 15.71$ mm - At F500 mm/min: Time = $15.71 / 500 = 0.0314$ min = 1.88 seconds

Plane Selection

G17, G18, G19

Circular interpolation requires a plane selection:

Code	Plane	Arc Axes	Perpendicular Axis
G17	XY	X, Y (I, J)	Z
G18	XZ	X, Z (I, K)	Y
G19	YZ	Y, Z (J, K)	X

Example G17 (default):

```
G17                                (XY plane)
G02 X10 Y10 I5 J0 F500    (Arc in XY, I and J offsets)
```

Example G18:

G18 (XZ plane)
G02 X10 Z-5 I5 K0 F500 (Arc in XZ, I and K offsets)

Example G19:

G19 (YZ plane)
G02 Y10 Z-5 J5 K0 F500 (Arc in YZ, J and K offsets)

Default: Most controls default to G17 at startup.

Feed Rate Modes

G94 - Feed Per Minute (Default)

Feed rate specifies tool velocity in units per minute:

G94 (Feed per minute mode)
G01 X100.0 F500 (500 mm/min or IPM)

Characteristics: - Most common mode - Feed rate independent of spindle speed - Consistent for both linear and circular motion

G95 - Feed Per Revolution

Feed rate specifies tool advance per spindle revolution:

G95 (Feed per revolution mode)
S1000 M03 (1000 RPM)
G01 X100.0 F0.1 (0.1 mm per revolution = 100 mm/min)

Effective feed rate:

$$F_{effective} = F_{per_rev} \times S_{RPM}$$

Applications: - Threading - Turning operations - Constant chip load regardless of diameter

G93 - Inverse Time Feed

Feed rate specifies the inverse of the time for the move:

G93 (Inverse time mode)
G01 X100.0 F10.0 (Move completes in 1/10 = 0.1 minutes)

Characteristics: - F-word must be specified for every move - Used in some CAM post-processors for 5-axis
- Ensures predictable move completion time

Motion Control Modes

G61 - Exact Stop Mode

The machine comes to a complete stop at each endpoint:

G61 (Exact stop mode)
G01 X10 Y10 F500
G01 X20 Y10
G01 X20 Y20

Each corner is a full deceleration to zero, then acceleration to next move.

Characteristics: - Guarantees corner accuracy - Slower cycle time due to decel/accel - Use for precise corners, inspection points

G64 - Continuous Path Mode

The machine blends motion between blocks without stopping:

```
G64 P0.01          (Continuous mode, 0.01mm path tolerance)
G01 X10 Y10 F500
G01 X20 Y10
G01 X20 Y20
```

Characteristics: - Maintains velocity through corners (path blending) - Faster cycle times - May round corners slightly (within tolerance P value) - Default mode on most modern controls

Tolerance parameter: - P value: Maximum deviation from programmed path - Smaller P = tighter corners, slower speed - Larger P = smoother motion, higher speed

G64 P0 - Maximum Speed Blending

```
G64 P0             (Maximum blending, no path tolerance limit)
```

Use for non-critical contouring where speed is prioritized over corner accuracy.

Practical Examples

Example 1: Simple Rectangle

```
G21 G90 G17 G94    (Metric, absolute, XY plane, feed/min)
G54                (Work offset 1)
T01 M06           (12mm end mill)
G43 H01           (Tool length offset)
S2000 M03         (Spindle on)

G00 X0 Y0 Z5.0     (Rapid to start, safe Z)
G01 Z-5.0 F100     (Plunge to depth)
G01 X50.0 Y0 F500  (Side 1)
G01 X50.0 Y25.0   (Side 2)
G01 X0 Y25.0      (Side 3)
G01 X0 Y0         (Side 4, close rectangle)
G01 Z5.0 F100     (Retract)

M05               (Spindle off)
M30               (Program end)
```

Example 2: Arc-Corner Rectangle

```
G21 G90 G17 G94
G54
T01 M06
G43 H01
S2000 M03

G00 X5.0 Y0 Z5.0   (Start 5mm in from corner)
G01 Z-5.0 F100     (Plunge)
G01 X45.0 Y0 F500  (Side 1)
G03 X50.0 Y5.0 R5.0 (Corner 1, radius 5)
G01 X50.0 Y20.0   (Side 2)
G03 X45.0 Y25.0 R5.0 (Corner 2)
G01 X5.0 Y25.0    (Side 3)
G03 X0 Y20.0 R5.0 (Corner 3)
```

```
G01 X0 Y5.0          (Side 4)
G03 X5.0 Y0 R5.0      (Corner 4, close)
G01 Z5.0 F100          (Retract)
```

```
M05
M30
```

Example 3: Circular Pocket (Helical Entry)

```
G21 G90 G17 G94
G54
T01 M06                (12mm end mill)
G43 H01
S2000 M03
```

```
(Approach center)
G00 X25.0 Y25.0 Z5.0    (Center of 50mm diameter pocket)
G01 Z0 F100              (Touch surface)
```

```
(Helical ramp entry)
G02 X25.0 Y25.0 Z-10.0 I-5.0 J0 F300    (Helix down 10mm, radius 5mm)
```

```
(Circular cuts at depth, spiraling outward)
G02 X25.0 Y25.0 I-10.0 J0 F500           (Circle radius 10mm)
G02 X25.0 Y25.0 I-15.0 J0               (Circle radius 15mm)
G02 X25.0 Y25.0 I-20.0 J0               (Circle radius 20mm)
G02 X25.0 Y25.0 I-23.0 J0               (Circle radius 23mm, finish)
```

```
(Retract)
G00 Z5.0
```

```
M05
M30
```

Troubleshooting Motion Commands

Common Errors

1. Feed rate not defined:

```
G01 X10 Y10          (ERROR if no F-word previously set)
```

Solution: Always initialize F-word before first G01.

2. Arc radius mismatch:

```
G02 X10 Y10 I3 J2    (May alarm if start/end radius differ)
```

Solution: Verify geometry, check CAM output, adjust tolerance.

3. Wrong plane selection:

```
G17                (XY plane)
G02 X10 Z10 I5 K0   (ERROR: Using X and Z in XY plane)
```

Solution: Switch to G18 for XZ arcs.

4. Full circle with R format:

```
G02 X10 Y0 R10      (ERROR: Start = end, ambiguous)
```

Solution: Use I, J, K format for 360 deg arcs.

Debugging Techniques

1. **Simulation:** Visualize toolpath in CAM or verifier
2. **Dry run:** Execute with feed override at 0% or in air above part
3. **Single block:** Step through program one line at a time
4. **Arc validation:** Calculate start/end radius manually to verify

Key Takeaways

1. **G00** is for rapid, non-cutting moves; **G01** is for linear cutting moves
 2. **Feed rate** (F-word) controls G01 velocity and is modal
 3. **G02/G03** create circular arcs; direction depends on plane and viewpoint
 4. **I, J, K** specify arc center as offset from start; **R** specifies radius
 5. **Plane selection** (G17/G18/G19) determines which axes participate in arcs
 6. **Helical interpolation** combines circular motion with linear Z movement
 7. **G64** enables path blending for smooth, fast contouring
 8. **Always validate** arc geometry and feed rates before running
-

Next: Section 15.4 - Coordinate Systems

Previous: Section 15.2 - G-Code Structure

Section 15.4 - Coordinate Systems and Work Offsets

Overview

Coordinate systems form the foundation of CNC positioning, enabling programs to reference part geometry independent of machine position. Work offsets, tool length compensation, and cutter radius compensation allow the same program to be used across multiple setups, tools, and machines.

This section covers absolute and incremental positioning, work coordinate systems (G54-G59), tool offsets, and the transformation hierarchy that connects machine coordinates to programmed part features.

Machine Coordinate System (MCS)

Definition

The **machine coordinate system** is the fundamental reference frame established when the machine is homed. It represents absolute positions relative to physical hard stops or encoder markers.

Characteristics: - Origin (0, 0, 0) at a fixed machine location - Established by homing sequence (G28, G30) - Never changes unless re-homed - Used internally by controller for limit checking

Typical machine origin locations:

Machine Type	X=0	Y=0	Z=0
Vertical mill	Left or center	Front or center	Top of travel
Horizontal mill	Spindle center	Table center	Spindle face
Lathe	Spindle centerline	-	Chuck face or tail
Gantry	Lower left	Lower left	Table surface

Homing Commands

G28 - Return to home via intermediate point:

G28 G91 Z0 (Return Z-axis to home via current position)
G28 X0 Y0 (Return X and Y to home via current position)

G28 workflow: 1. Move in incremental mode to specified intermediate point 2. From intermediate point, move to machine home 3. Reset internal position to machine zero

G30 - Return to secondary home (if equipped):

G30 G91 Z0 (Return Z to secondary home position)

Used for: - Tool changer position - Pallet change location - Safe parking position

Soft Limits

The machine coordinate system enforces travel limits:

(Machine X travel: -500 to +500)
G53 G00 X600 (ERROR: Beyond soft limit)

G53 accesses machine coordinates directly (bypass work offsets).

Work Coordinate Systems (WCS)

Purpose

Work offsets allow programming relative to part features rather than machine position. The same program can run on different fixtures, machines, or setups by changing only the offset values.

Example:

Program reference: Part corner at X0 Y0 Z0 - **Setup 1:** Part in vise at machine X100 Y50 Z-200 - **Setup 2:** Part in fixture at machine X300 Y150 Z-180

By setting work offsets, the program X0 Y0 Z0 automatically maps to the correct machine position.

G54 through G59 - Work Offset Selection

ISO 6983 defines six standard work coordinate systems:

Code	Work Coordinate System
G54	Work offset 1 (default)
G55	Work offset 2
G56	Work offset 3
G57	Work offset 4
G58	Work offset 5
G59	Work offset 6

Extended offsets (control-dependent): - **G59.1** through **G59.9:** Additional offsets - Some controls support G54.1 P1-P99 for 99+ offsets

Usage:

G54 (Select work offset 1)
G00 X0 Y0 Z0 (Move to part origin in G54)

G55 (Select work offset 2)
G00 X0 Y0 Z0 (Move to part origin in G55)

Setting Work Offsets

Work offsets are set through controller interface or G10 command:

Method 1: Manual (touch-off): 1. Jog tool to known part feature (corner, edge, hole center) 2. Record machine position 3. Enter offset: Machine_Position - Desired_Work_Position

Example: - Touch top of part with tool tip - Machine Z position: -187.325 - Desired work Z position: 0 (top of part) - G54 Z offset: -187.325 - 0 = -187.325

Method 2: Probing (automatic):

G54 (Select work offset to set)
G10 L20 P1 X0 Y0 (Set current position as X0 Y0 in G54)

Method 3: G10 L2 (set offset value):

G10 L2 P1 X100.0 Y50.0 Z-200.0 (Set G54 offsets directly)

Where P specifies the offset number (P1=G54, P2=G55, etc.)

Work Offset Table

Internally, the controller stores offset values:

Offset	X	Y	Z	Notes
G54	100.000	50.000	-200.000	Vise, jaw 1
G55	300.000	50.000	-200.000	Vise, jaw 2

Offset	X	Y	Z	Notes
G56	150.000	150.000	-180.000	Fixture plate A
G57	0	0	0	(not set)
G58	0	0	0	(not set)
G59	0	0	0	(not set)

Coordinate Transformation

When work offset is active, programmed coordinates transform:

$$Machine_Position = Work_Position + Offset_Value$$

Example: - Active offset: G54 (X100, Y50, Z-200) - Programmed move: G01 X10 Y20 Z-5 - Machine moves to: X110, Y70, Z-205

G53 - Machine Coordinate Override

G53 bypasses all offsets for a single block:

```
G54                (Work offset active)
G53 G00 X0 Y0 Z0   (Move to machine home, ignore G54)
G00 X0 Y0 Z0       (Move to work offset origin)
```

Applications: - Tool changer position - Safe parking location - Absolute machine moves for setup

Important: G53 is non-modal (one block only).

Absolute and Incremental Positioning

G90 - Absolute Mode (Default)

In absolute mode, coordinates specify the endpoint position relative to the work coordinate system origin.

```
G90                (Absolute mode)
G54                (Work offset 1)
G01 X10 Y10 F500   (Move to X=10, Y=10 in G54)
G01 X20 Y20        (Move to X=20, Y=20 in G54)
```

Characteristics: - Most common programming mode - Coordinates match part print dimensions - Easy to verify and troubleshoot - Cumulative errors do not occur

G91 - Incremental Mode

In incremental mode, coordinates specify movement distance from the current position.

```
G91                (Incremental mode)
G01 X10 Y10 F500   (Move +10 in X, +10 in Y from current)
G01 X10 Y10        (Move another +10 in X, +10 in Y)
```

Characteristics: - Used for repetitive patterns, bolt circles - Easier for some pocketing operations - Risk of cumulative positioning errors - Harder to verify against print

Modal Behavior

Distance mode (G90/G91) is modal and persists until changed:

```

G90                (Absolute mode active)
G01 X10 Y10
G01 X20 Y20        (Still absolute)
G91                (Switch to incremental)
G01 X5 Y5          (Incremental move)
G01 X5 Y5          (Still incremental)
G90                (Return to absolute)

```

Best practice: Always initialize G90 or G91 at program start.

Mixed Mode (G90.1, G91.1)

Some controls support independent distance modes for arcs:

```

G90 G91.1          (Absolute XYZ, incremental IJK)
G02 X20 Y20 I5 J0  (Endpoint absolute, center offset incremental)

```

G90.1: Arc center (IJK) in absolute mode **G91.1:** Arc center (IJK) in incremental mode (default)

Tool Length Compensation

Purpose

Tool length offset compensates for differences in tool length, allowing the program to reference part geometry (e.g., top surface Z=0) regardless of which tool is in the spindle.

Without tool offset: - Program must know exact tool length - Changing tools requires program edits - Impractical for multi-tool operations

With tool offset: - Program references part geometry - Tool table stores each tool's length - Tool changes require no program edits

G43 - Tool Length Offset Enable

G43 activates tool length compensation:

```

T01 M06            (Change to tool 1)
G43 H01            (Apply tool length offset 1)
G00 Z0             (Move to Z=0 in work coordinates)

```

H-word specifies the offset register (usually matches tool number).

G49 - Tool Length Offset Cancel

G49 cancels tool length offset:

```

G49                (Cancel tool length offset)
G00 Z0             (Z=0 in machine coordinates, dangerous!)

```

Warning: G49 with Z moves can crash tool into table. Always retract Z before G49.

Tool Length Offset Table

Controller stores measured tool lengths:

Tool	H	Length (mm)	Description
T01	H01	150.325	12mm end mill
T02	H02	175.128	6mm drill
T03	H03	148.956	8mm end mill
T04	H04	165.442	Spot drill

Tool	H	Length (mm)	Description
------	---	-------------	-------------

Setting Tool Lengths

Method 1: Touch-off to known surface: 1. Load tool in spindle 2. Jog tool tip to touch reference surface (gauge block, part top) 3. Note machine Z position 4. Calculate: Length = Reference_Height - Machine_Z

Method 2: Tool height setter (automatic): - Tool touches probe on table - Controller measures and stores length - Reference height already known

Method 3: G10 L1 (set tool length directly):

G10 L1 P1 Z150.325 (Set tool 1 length to 150.325)

Tool Length Calculation Example

Scenario: - Reference surface: Top of part, Z=0 in work coordinates - Machine Z when touching part: -187.325 - Tool tip at part top: -187.325 (machine) = 0 (work)

With tool change: - Tool 1 length: 150.325 - Tool 2 length: 175.128 - Difference: 24.803

When switching from T01 to T02: - Controller automatically adjusts Z position by -24.803 - Tool tip remains at same work coordinate Z position

G43.1 - Dynamic Tool Length Offset

Some controls support dynamic offset specification:

G43.1 Z150.325 (Apply offset value directly)

Used for: - Temporary offsets - Parametric tool length from variables - Special probing operations

Cutter Radius Compensation

Purpose

Cutter radius compensation adjusts the toolpath to account for tool radius, allowing programs to be written to part geometry rather than tool center.

Without compensation: - Program must offset path by tool radius - Changing tool size requires program edits - CAM must generate tool center path

With compensation: - Program follows part edge - Tool radius in tool table - Same program works with different tool sizes

G40, G41, G42

Code	Function
G40	Cutter compensation off
G41	Cutter compensation left
G42	Cutter compensation right

G41 (left): Tool moves to left of programmed path (for outside profile) **G42 (right):** Tool moves to right of programmed path (for inside profile)

Direction is determined by looking in the direction of travel.

Activation and Cancellation

Activate:

```
G00 X-10 Y0      (Approach outside part)
G41 D01 G01 X0 Y0 F500 (Enable compensation, move to part edge)
G01 X50 Y0       (Follow part edge, compensated)
G01 X50 Y25
G01 X0 Y25
G01 X0 Y0
G40 G01 X-10 Y0   (Cancel compensation, move away)
```

D-word specifies the tool radius offset register.

Cancel:

```
G40              (Cancel compensation)
```

Tool Radius Table

Tool	D	Radius (mm)	Diameter (mm)
T01	D01	6.000	12.0
T02	D02	3.000	6.0
T03	D03	4.000	8.0

Compensation Logic

The controller offsets the tool path perpendicular to the direction of travel:

Example: - Programmed path: X0->X50 (horizontal line) - Tool radius: 6mm - G41 active: Tool center moves from Y-6->Y-6 (tool below path) - G42 active: Tool center moves from Y+6->Y+6 (tool above path)

Lead-In and Lead-Out

Compensation requires lead-in and lead-out moves:

```
(Outside profile - G41)
G00 X-10 Y0      (Start outside part)
G41 D01 G01 X0 Y0 (Lead-in, activate compensation)
(... part profile ...)
G40 G01 X-10 Y0   (Lead-out, cancel compensation)
```

Lead-in distance: At least 1x tool radius, preferably 2-3x

Look-Ahead

The controller looks ahead to future blocks to calculate correct offset:

```
G41 D01          (Compensation active)
G01 X10 Y0       (Straight)
G01 X10 Y10      (90 deg corner)
```

At the corner, controller automatically calculates the circular arc to maintain constant offset from both edges.

G41.1 / G42.1 - Dynamic Compensation

```
G41.1 D6.0       (Activate compensation with radius 6mm directly)
```

Useful for parametric programming with variable tool sizes.

Coordinate System Priority

Transformation Hierarchy

Coordinates are transformed through multiple systems:

1. **Programmed coordinate** (as written in G-code)
2. + **Work offset** (G54-G59)
3. + **Tool length offset** (G43 H___)
4. + **Cutter radius compensation** (G41/G42 D___)
5. = **Machine coordinate** (actual axis position)

Example: - Program: G01 Z-10 (cut 10mm deep) - G54 Z offset: -200.0 (part top at machine Z-200) - Tool length H01: 150.325 - Machine Z: $-10 + (-200.0) + 150.325 = \mathbf{-59.675}$

Order of Operations

Commands must be issued in correct sequence:

Correct:

```
G54           (Select work offset)
T01 M06      (Change tool)
G43 H01      (Apply tool length)
G00 X0 Y0    (Position in work coordinates)
```

Incorrect:

```
G00 X0 Y0    (Move where? No work offset or tool length set!)
G54          (Too late)
G43 H01      (Tool length applied after move)
```

G92 - Work Coordinate System Shift

Function

G92 temporarily shifts the work coordinate system without changing stored offsets.

```
G92 X0 Y0 Z0      (Set current position as origin)
```

Effect: Current machine position becomes the specified work coordinate.

Use Cases

Temporary origin shift:

```
G54           (Work offset 1)
G00 X50 Y50    (Move to secondary feature)
G92 X0 Y0      (Treat this position as new origin)
(... machine operations relative to X50 Y50 ...)
G92.1         (Clear G92 shift, restore original G54)
```

Legacy machines: - Some older controls use G92 instead of G54-G59 - Common on hobbyist/entry-level machines

G92 vs. G54

Feature	G54-G59	G92
Storage	Permanent (in offset table)	Temporary (cleared at reset)

Feature	G54-G59	G92
Multiple offsets	Yes (6+)	No (single shift)
Recommended	Yes (modern practice)	No (legacy only)

Best practice: Avoid G92 if G54-G59 are available.

G92.1, G92.2, G92.3

- **G92.1:** Cancel G92 offset, restore to G54-G59
- **G92.2:** Suspend G92 offset temporarily
- **G92.3:** Restore suspended G92 offset

Practical Examples

Example 1: Multi-Part Fixture

(Program machines 3 identical parts in different vise locations)

```
O1000          (Main program)
G54 M98 P1100   (Part 1 in G54, call subprogram)
G55 M98 P1100   (Part 2 in G55, call subprogram)
G56 M98 P1100   (Part 3 in G56, call subprogram)
M30            (End main)
```

```
O1100          (Subprogram - part operations)
G00 X0 Y0 Z5.0  (Rapid to origin)
G01 Z-5.0 F100  (Plunge)
(... machining operations ...)
G00 Z5.0        (Retract)
M99            (Return to main)
```

Example 2: Tool Change with Offsets

(--- TOOL 1: 12MM END MILL ---)

```
T01 M06
G43 H01          (Tool length offset 1)
S2000 M03
G54             (Part 1 work offset)
G00 X0 Y0 Z5.0
(... machining ...)
```

(--- TOOL 2: 6MM DRILL ---)

```
M05
G00 Z50.0
T02 M06
G43 H02          (Tool length offset 2 - automatically adjusts Z)
S3000 M03
(Tool tip remains at same work Z position despite length difference)
G00 X10 Y10 Z5.0
(... drilling ...)
```

Example 3: Cutter Compensation Profile

(12mm end mill, radius 6mm)

G21 G90 G17

G54

T01 M06

G43 H01

S2000 M03

(Approach and lead-in)

G00 X-10 Y12.5 Z5.0

G01 Z-5.0 F100

(Enable compensation, machine outside profile)

G41 D01 G01 X0 Y12.5 F500 (Lead-in to part edge)

G01 X50.0 (Bottom edge)

G01 Y25.0 (Right edge)

G01 X0 (Top edge)

G01 Y12.5 (Left edge, close)

G40 G01 X-10.0 (Cancel compensation, lead-out)

G00 Z50.0

M05

M30

Key Takeaways

1. **Machine coordinates** are absolute reference; **work offsets** enable part-relative programming
2. **G54-G59** provide six standard work coordinate systems for multiple setups
3. **G90** (absolute) is standard; **G91** (incremental) for special cases
4. **Tool length compensation** (G43) allows part-referenced Z programming
5. **Cutter radius compensation** (G41/G42) offsets path for tool diameter
6. **Transformation hierarchy**: Program -> Work Offset -> Tool Length -> Machine
7. **G53** bypasses offsets for direct machine coordinate access
8. **Proper initialization** of offsets and tool compensation is critical for safe operation

Next: Section 15.5 - Auxiliary Functions

Previous: Section 15.3 - Motion Commands

Section 15.5 - Auxiliary Functions (M-Codes)

Overview

M-codes (Miscellaneous functions) control auxiliary machine functions beyond motion: spindle rotation, coolant activation, tool changes, program flow, and machine-specific operations. Unlike preparatory functions (G-codes) that primarily control motion, M-codes interact with peripheral systems.

This section covers standard M-codes, their syntax, modal behavior, and the sequence in which they execute relative to motion commands.

M-Code Fundamentals

Syntax

M-codes follow the same word format as G-codes:

```
M03      (Letter M + numeric code)
M08      (Most M-codes are two digits)
M30      (Some controls support M100-M999 for custom macros)
```

Execution Timing

M-codes execute at specific points in block processing:

Type 1: Before motion - M-code executes, then motion begins - Example: M03 (spindle on) starts before G01 move

Type 2: After motion - Motion completes, then M-code executes - Example: M00 (program stop) after move to position

Type 3: Immediate - M-code executes instantly, independent of motion - Example: M30 (program end) terminates immediately

Modal vs. Non-Modal

Most M-codes are **non-modal** (execute once):

```
M03      (Spindle on, stays on until turned off)
```

Some M-codes are **modal** and remain active:

```
M08      (Coolant on, stays on)
M09      (Coolant off)
```

Multiple M-Codes Per Block

ISO 6983 limits blocks to **one M-code** per line. Some modern controls allow multiple if they don't conflict:

Standard (safe):

```
M03      (Spindle on)
M08      (Coolant on - separate block)
```

Some controls allow:

```
M03 M08  (Spindle and coolant on - check manual)
```

Best practice: One M-code per block for maximum compatibility.

Spindle Control

M03 - Spindle On Clockwise (CW)

Starts spindle rotation in the clockwise direction (viewed from spindle nose toward motor).

S2000 M03 (Spindle on at 2000 RPM clockwise)

Characteristics: - Requires S-word for spindle speed (RPM) - Waits for spindle to reach speed (if control monitors RPM) - Modal: remains on until M05 or M04

Typical applications: - Milling (most operations) - Drilling - Right-hand thread cutting

M04 - Spindle On Counterclockwise (CCW)

Starts spindle rotation counterclockwise:

S2000 M04 (Spindle on at 2000 RPM counterclockwise)

Typical applications: - Left-hand thread tapping - Reverse facing operations (lathes) - Spindle cleaning / chip evacuation

Safety: Verify tool rotation direction before use. Right-hand tools with M04 can self-loosen.

M05 - Spindle Stop

Stops spindle rotation:

M05 (Spindle off)

Characteristics: - Non-modal (executes once) - Controller may wait for spindle to reach zero RPM - Always issue before tool change

Standard sequence:

M05 (Stop spindle)
G04 P2.0 (Dwell 2 seconds for spindle to stop)
T02 M06 (Safe to change tool)

S-Word - Spindle Speed

The S-word specifies spindle speed in RPM:

S3000 (Set spindle speed to 3000 RPM)
M03 (Start spindle at 3000 RPM)

Modal behavior: - S-word is modal (remains active) - Can be changed during operation:

S2000 M03 (Start at 2000 RPM)
G01 X50 F500 (Cutting move)
S2500 (Increase to 2500 RPM mid-cut)

Speed limits: - Controlled by machine parameters - Typical ranges: 100-8000 RPM (milling), 10-50,000 RPM (spindles) - Controller enforces maximum speed limits

M19 - Spindle Orientation

Orients spindle to a specific angular position:

M19 (Orient spindle to 0 deg - control-specific)
M19 P90 (Orient to 90 deg on some controls)

Applications: - Tool change orientation for automatic tool changers - Angle head positioning - Spindle probe mounting

Coolant Control

M08 - Coolant On

Activates flood coolant:

M08 (Flood coolant on)

Characteristics: - Modal: remains on until M09 - Usually activates primary coolant pump - May activate multiple coolant nozzles

Standard usage:

S2000 M03 (Spindle on)
M08 (Coolant on)
G01 Z-10 F100 (Begin cutting with coolant)

M09 - Coolant Off

Deactivates coolant:

M09 (Coolant off)

Standard usage:

G00 Z50 (Retract to safe height)
M09 (Coolant off)
M05 (Spindle off)

M07 - Mist Coolant On

Activates mist coolant (if equipped):

M07 (Mist coolant on)

Characteristics: - Used for air-oil mist or air blast - Often for high-speed finishing - Machine-dependent implementation

Combined coolant:

M07 M08 (Both mist and flood - if control allows)

M88/M89 - Through-Spindle Coolant (TSC)

Some machines support coolant through tool center:

M88 (TSC on - control-specific)
M89 (TSC off - control-specific)

Applications: - Deep hole drilling - Gun drilling - High-pressure peck drilling

Pressure requirements: - Typically 300-1000 PSI (20-70 bar) - Requires special tooling with coolant passages

Tool Change

M06 - Tool Change

Executes automatic tool change:

T05 M06 (Change to tool 5)

Sequence: 1. Spindle stops (if not already stopped) 2. Z-axis retracts to tool change position 3. Tool changer swaps tools 4. Spindle returns to ready position

Standard tool change block:

```
M05          (Stop spindle)
M09          (Coolant off)
G28 G91 Z0   (Retract Z to home)
T02 M06      (Change to tool 2)
G43 H02      (Apply new tool length offset)
S3000 M03    (Start spindle at new speed)
M08          (Coolant on)
```

T-Word - Tool Selection

The T-word specifies tool number:

```
T01          (Prepare tool 1 for change)
M06          (Execute tool change)
```

Pre-selection (some controls):

```
T02          (Pre-select tool 2)
(... machining with tool 1 ...)
M06          (Change to pre-selected tool 2)
```

Pre-selection reduces tool change time on machines with tool magazines.

M61 - Set Tool Number

Sets current tool number without physical change:

```
M61 Q5       (Tell control tool 5 is in spindle)
```

Use cases: - Manual tool changes - Tool broke, replaced with same type - Program restart after interruption

Program Control

M00 - Program Stop

Stops program execution and waits for operator input:

```
G00 X50 Y50
M00          (Stop here, wait for cycle start)
G01 Z-10 F100 (Continues after operator presses start)
```

Behavior: - Motion stops - Spindle and coolant remain on (usually) - Operator must press Cycle Start to continue - Use for inspection, measurement, chip clearing

M01 - Optional Program Stop

Conditional stop, active only if “Optional Stop” switch is enabled:

```
G00 X50 Y50
M01          (Stop only if optional stop enabled)
G01 Z-10 F100
```

Use cases: - Inspection points for first article - Chip clearing for long programs - Coolant check - Can be disabled for production runs

M02 - Program End

Ends program execution:

M02 (Program end)

Behavior: - Stops motion - Does NOT stop spindle or coolant (machine-dependent) - Does NOT rewind program - Less common than M30

M30 - Program End and Rewind

Ends program and resets to beginning:

M30 (Program end and rewind)

Behavior: - Stops motion - Stops spindle (M05) - Stops coolant (M09) - Rewinds program to beginning - Resets some modal states (machine-dependent)

Standard program ending:

G00 Z50.0 (Retract)
M09 (Coolant off)
M05 (Spindle off)
G28 G91 Z0 (Home Z)
G28 X0 Y0 (Home XY)
G90 (Restore absolute mode)
M30 (End and rewind)

M98 - Subprogram Call

Calls an external subprogram:

M98 P1234 (Call program 01234)
M98 P1234 L5 (Call 01234, repeat 5 times)

Syntax variations: - **P-word:** Subprogram number - **L-word:** Repeat count (optional, default = 1)

Subprogram structure:

01234 (Subprogram number)
G01 X10 Y10 F500
G01 X20 Y20
M99 (Return to main program)

M99 - Return from Subprogram

Returns control to calling program:

M99 (Return from subprogram)
M99 P5000 (Jump to block N5000 in main program)

Behavior: - Resumes at block following M98 call - Can specify return address with P-word (control-dependent)

Machine-Specific M-Codes

Common Extensions

Many controls define custom M-codes beyond ISO 6983:

M-Code	Function	Notes
M10	Clamp engaged	4th axis, pallet
M11	Clamp released	4th axis, pallet
M21-M28	Mirror image on/off	Axis-specific
M50-M59	Custom coolant zones	Multi-nozzle
M60	Pallet change	Horizontal machining centers
M70-M72	Save/restore modal state	Program nesting
M98-M99	Subprogram call/return	Universal
M100+	Custom macros	User-defined

LinuxCNC Custom M-Codes

LinuxCNC supports custom M-codes via external scripts:

Example M101 (custom coolant):

```
#!/bin/bash
# File: M101
# Activate high-pressure coolant
echo "Activating high-pressure coolant"
hal setp coolant-hp.enable true

M101          (Calls /usr/local/bin/M101 script)
```

Probing M-Codes (G38.x alternative)

Some controls use M-codes for probing:

```
M75          (Enable probe)
G38.2 Z-50 F50 (Probe toward Z-50)
M76          (Disable probe)
```

Air and Chip Management

M88/M89 - Air Blast

On some machines, M88/M89 control air blast:

```
M88          (Air blast on)
M89          (Air blast off)
```

Applications: - Chip clearing during drilling - Part cleaning - Dry machining assist

M35/M36 - Chip Conveyor

Chip conveyor control (machine-dependent):

```
M35          (Chip conveyor forward)
M36          (Chip conveyor reverse)
M37          (Chip conveyor stop)
```

Execution Examples

Example 1: Basic Spindle and Coolant

```
G21 G90 G17 G54          (Initialize)
T01 M06                  (Tool 1)
G43 H01                  (Tool length offset)
```

S2000 M03	(Spindle on 2000 RPM clockwise)
G04 P2.0	(Dwell 2 seconds for spindle to reach speed)
M08	(Coolant on)
G00 X0 Y0 Z5.0	(Position)
G01 Z-10.0 F100	(Plunge with coolant)
G01 X50.0 F500	(Cut)
G00 Z50.0	(Retract)
M09	(Coolant off)
M05	(Spindle off)
M30	(End program)

Example 2: Tool Change Sequence

```

(--- TOOL 1: ROUGHING ---)
T01 M06
G43 H01
S2000 M03
M08
G00 X0 Y0 Z5.0
(...machining operations...)

(--- TOOL CHANGE TO TOOL 2 ---)
G00 Z50.0          (Retract)
M09                (Coolant off)
M05                (Spindle stop)
G04 P3.0           (Dwell for spindle stop)
T02 M06            (Change to tool 2)

(--- TOOL 2: FINISHING ---)
G43 H02            (New tool length)
S3000 M03          (Higher speed for finishing)
M08                (Coolant back on)
G00 X0 Y0 Z5.0
(...finishing operations...)

M09
M05
M30

```

Example 3: Optional Stop for Inspection

```

G21 G90 G17 G54
T01 M06
G43 H01
S2000 M03
M08

(Rough pass)
G00 X0 Y0 Z5.0
G01 Z-10.0 F100
G01 X50.0 F500

```

```

M01                      (Optional stop - inspect if switch enabled)

(Finish pass)
G01 Z-10.2 F100          (Deeper for finish)
G01 X50.0 F300           (Slower feed for finish)

G00 Z50.0
M09
M05
M30

```

Example 4: Subprogram for Bolt Circle

```

(Main program)
O1000
G21 G90 G17 G54
T01 M06
G43 H01
S3000 M03
M08

G00 X50.0 Y50.0 Z5.0      (Center of bolt circle)

M98 P2000 L4              (Call hole subprogram 4 times)

G00 Z50.0
M09
M05
M30

(Subprogram - drill one hole and rotate)
O2000
G81 X10.0 Y0 Z-10.0 R2.0 F100  (Drill hole at 0 deg)
G00 G91 A90.0                (Rotate 90 deg incremental)
G90                          (Back to absolute)
M99                          (Return)

```

M-Code Timing and Safety

Dwell for Spindle Stabilization

Always allow spindle to reach commanded speed:

```

S5000 M03                (Start 5000 RPM spindle)
G04 P3.0                 (Wait 3 seconds)
G01 Z-10 F100            (Safe to begin cutting)

```

Typical dwell times: - Low RPM (< 2000): 1-2 seconds - Medium RPM (2000-5000): 2-3 seconds - High RPM (> 5000): 3-5 seconds

Coolant Before Spindle

To prevent dry start:

```

M08                      (Coolant on first)
G04 P1.0                 (Brief delay)
S2000 M03                (Spindle on with coolant flowing)

```

Safe Tool Change

Standard safe sequence:

G00 Z50.0	(Retract to safe Z)
M09	(Coolant off)
M05	(Spindle off)
G04 P3.0	(Wait for spindle stop)
G28 G91 Z0	(Home Z-axis)
T02 M06	(Now safe to change tool)

Control-Specific Variations

FANUC

M03 S2000	(S before or after M03)
M98 P8500	(Subprogram call, local)
M198 P8500	(Subprogram call, external)

Siemens

M3 S2000	(No leading zero)
M17	(End of subprogram)
M02	(Program end, more common than M30)

Heidenhain

M3	(No leading zero)
M6	(Tool change)
M91	(Activate subprogram)

LinuxCNC

M03 S2000	(Standard)
M64 P0	(Set digital output bit 0)
M65 P0	(Clear digital output bit 0)
M66 P0 L1	(Wait for digital input bit 0)

Key Takeaways

1. **M-codes control auxiliary functions:** spindle, coolant, tool changes, program flow
2. **M03/M04** start spindle CW/CCW; **M05** stops spindle
3. **M08** coolant on; **M09** coolant off
4. **M06** tool change with T-word for tool selection
5. **M00** program stop; **M01** optional stop; **M30** end and rewind
6. **M98/M99** subprogram call and return
7. **Timing is critical:** dwell after spindle start, coolant before cutting
8. **One M-code per block** for maximum compatibility
9. **Machine-specific M-codes** extend functionality beyond ISO 6983 standard

Next: Section 15.6 - Canned Cycles

Previous: Section 15.4 - Coordinate Systems

Section 15.6 - Canned Cycles

Overview

Canned cycles are single-block commands that execute complex, repetitive machining operations automatically. Instead of programming every motion explicitly, a canned cycle condenses drilling, tapping, boring, and pocketing operations into one line of code with parameters.

This section covers the standard canned cycles (G81-G89), their parameters, return modes, and practical applications for efficient CNC programming.

Canned Cycle Fundamentals

Purpose and Benefits

Without canned cycles:

```
G00 X10 Y10 Z5.0      (Position above hole 1)
G01 Z-20.0 F100        (Drill to depth)
G00 Z5.0               (Retract)
G00 X20 Y10            (Position above hole 2)
G01 Z-20.0 F100        (Drill to depth)
G00 Z5.0               (Retract)
(... repeat for every hole ...)
```

With canned cycles:

```
G81 X10 Y10 Z-20.0 R5.0 F100  (Drill hole 1)
X20 Y10                       (Drill hole 2)
X30 Y10                       (Drill hole 3)
G80                           (Cancel cycle)
```

Benefits: - **Reduced code length:** 1-2 lines per hole vs. 5-10 lines - **Fewer errors:** Standardized motion sequence - **Easier editing:** Change depth or feed once, affects all holes - **Better readability:** Intent is clear from G-code number

Common Parameters

All canned cycles use a standard set of parameters:

Parameter	Meaning	Units
X, Y	Hole position (in current work offset)	Distance
Z	Depth of operation (absolute or incremental)	Distance
R	Retract plane (safe height above surface)	Distance
Q	Peck increment (for peck drilling)	Distance
P	Dwell time at bottom of hole	Seconds or milliseconds
F	Feed rate for drilling/boring	Units/min

Canned Cycle Sequence

Standard motion sequence for most cycles:

1. **Rapid (G00) to XY position** at current Z height
2. **Rapid (G00) to R plane** (retract height)
3. **Feed (G01) operation** specific to cycle type
4. **Action at depth** (dwell, stop, reverse)
5. **Retract** to R plane or initial Z (depending on return mode)
6. **Repeat** for next XY position

Modal Behavior

Canned cycles are **modal** - once activated, they repeat at each new XY position:

```
G81 X10 Y10 Z-20 R5.0 F100    (Activate cycle, drill hole 1)
X20 Y10                      (Drill hole 2 with same parameters)
X30 Y10                      (Drill hole 3)
Y20                          (Drill hole 4 at X30 Y20)
G80                          (Cancel cycle)
```

G80 - Cancel Canned Cycle

G80 cancels active canned cycle:

```
G80                          (Cancel canned cycle mode)
```

After G80, XY moves do not trigger drilling operations.

Best practice: Always cancel canned cycles before resuming normal programming.

Return Modes

G98 - Return to Initial Z

G98 returns the tool to the Z height before the cycle started:

```
G00 Z10.0                    (Start at Z=10)
G98 G81 X10 Y10 Z-20 R5.0 F100 (Drill, return to Z=10)
X20 Y10                      (Drill, return to Z=10)
```

Motion: 1. Rapid to X10 Y10 at Z=10 2. Rapid to R plane (Z=5) 3. Feed to depth (Z=-20) 4. Rapid back to **Z=10 (initial level)** 5. Rapid to X20 Y10 at Z=10 6. Repeat cycle

Use when: - Holes are far apart - Obstacles between holes (clamps, risers) - Maximum clearance needed

G99 - Return to R Plane

G99 returns the tool to the R plane only:

```
G00 Z10.0                    (Start at Z=10)
G99 G81 X10 Y10 Z-20 R5.0 F100 (Drill, return to R=5)
X20 Y10                      (Drill, return to R=5)
```

Motion: 1. Rapid to X10 Y10 at Z=10 2. Rapid to R plane (Z=5) 3. Feed to depth (Z=-20) 4. Rapid back to **Z=5 (R plane)** 5. Rapid to X20 Y10 at Z=5 6. Repeat cycle

Use when: - Holes are close together - No obstacles between holes - Faster cycle time (less Z travel)

Default: Most controls default to G99 (R-plane return).

Standard Drilling Cycles

G81 - Drilling Cycle (Simple)

Basic drilling cycle with feed in, rapid out:

```
G81 X__ Y__ Z__ R__ F__
```

Sequence: 1. Rapid to XY at current Z 2. Rapid to R plane 3. Feed to Z depth 4. Rapid out to R plane (G99) or initial Z (G98)

Example:

```
G00 Z10.0           (Initial Z)
G99 G81 X10 Y10 Z-20 R5.0 F100 (Drill to -20mm depth)
X20 Y10             (Drill second hole)
X30 Y10             (Drill third hole)
G80                 (Cancel)
```

Applications: - Through holes in thin material - Spotting for larger drills - Pilot holes

Limitations: - No chip breaking (can pack chips in deep holes) - Full-depth feed (may overload small drills)

G82 - Drilling Cycle with Dwell

Drilling with dwell at bottom for chip breaking and hole finish:

```
G82 X__ Y__ Z__ R__ P__ F__
```

Additional parameter: - **P:** Dwell time at depth (seconds or milliseconds, control-dependent)

Sequence: 1. Rapid to XY at current Z 2. Rapid to R plane 3. Feed to Z depth 4. **Dwell for P seconds** 5. Rapid out

Example:

```
G99 G82 X10 Y10 Z-20 R5.0 P1.0 F100   (Drill with 1-second dwell)
X20 Y10
G80
```

Applications: - Flat-bottom holes - Breaking stringy chips (aluminum, brass) - Improved hole finish - Chamfering hole exit

Dwell time selection: - **Chip breaking:** 0.5-1.0 seconds - **Finish improvement:** 1.0-2.0 seconds - **Too long:** Wastes time, may work-harden material

G83 - Peck Drilling Cycle

Deep hole drilling with periodic retraction for chip clearing:

```
G83 X__ Y__ Z__ R__ Q__ F__
```

Additional parameter: - **Q:** Peck increment (depth to drill before retract)

Sequence (each peck): 1. Rapid to XY at current Z (first peck only) 2. Rapid to R plane (first peck only) 3. Feed down Q distance (or to Z depth, whichever is less) 4. **Rapid retract to R plane (full retract)** 5. Rapid back to 0.1mm above previous depth 6. Repeat until Z depth reached 7. Rapid out

Example:

```
G99 G83 X10 Y10 Z-50 R5.0 Q10.0 F80   (Drill 50mm deep, 10mm pecks)
X20 Y10
G80
```

Motion detail for Q10, Z-50: - Peck 1: Feed to -10, rapid to R5.0, rapid to -9.9 - Peck 2: Feed to -20, rapid to R5.0, rapid to -19.9 - Peck 3: Feed to -30, rapid to R5.0, rapid to -29.9 - Peck 4: Feed to -40, rapid to R5.0, rapid to -39.9 - Peck 5: Feed to -50, rapid out

Applications: - Deep holes (depth > 3x diameter) - Difficult-to-machine materials - Prevents chip packing and tool breakage

Q value selection:

$$Q = (1.5 \text{ to } 3) \times D$$

Where D = drill diameter

- **Aluminum, soft materials:** $Q = 3D$
- **Steel, hard materials:** $Q = 1.5-2D$
- **Very deep holes:** Start with 2D, reduce if problems

G73 - High-Speed Peck Drilling

Similar to G83 but with **partial retract** for chip breaking:

G73 X__ Y__ Z__ R__ Q__ F__

Sequence (each peck): 1. Feed down Q distance 2. **Rapid retract 0.5-1.0mm (partial retract)** 3. Repeat until Z depth reached 4. Rapid out to R plane

Difference from G83: - G83: Full retract to R plane (slow, thorough chip clearing) - G73: Partial retract (fast, chip breaking only)

Applications: - Holes 3-8x diameter deep - Cast iron (short chips) - High-speed drilling - When full retract is unnecessary

Example:

G99 G73 X10 Y10 Z-30 R5.0 Q8.0 F150 (Fast peck, 30mm deep)

Tapping Cycles

G84 - Tapping Cycle (Synchronized)

Rigid tapping with synchronized spindle and Z-axis:

G84 X__ Y__ Z__ R__ F__

Feed rate calculation:

$$F = S \times P$$

Where: - F = feed rate (mm/min or IPM) - S = spindle speed (RPM) - P = thread pitch (mm or inches per thread)

Sequence: 1. Rapid to XY at current Z 2. Rapid to R plane 3. **Spindle on CW (M03)** 4. Feed to Z depth (synchronized) 5. **Spindle reverse CCW (M04)** 6. Feed out to R plane (synchronized) 7. **Spindle CW (M03)**

Example (M6x1.0 thread):

S500 M03 (500 RPM)
 G99 G84 X10 Y10 Z-20 R5.0 F500 (F = 500 RPM x 1.0 mm/rev)
 X20 Y10
 G80
 M05

For inch threads:

S400 M03 (400 RPM)
 G99 G84 X1.0 Y1.0 Z-0.75 R0.2 F100 (1/4-20: F = 400 x 0.05 = 20 IPM)

Where 1/20 = 0.05 inches per thread

Requirements: - Rigid tapping capability (encoder feedback) - Proper feed rate calculation - Tapping head or tension/compression holder (for floating taps)

Applications: - Through holes and blind holes - Consistent thread depth - Faster than hand tapping

G74 - Left-Hand Tapping (Counter-boring)

Tapping cycle for left-hand threads or counter-boring:

G74 X__ Y__ Z__ R__ F__

Sequence: - Same as G84, but starts with **M04 (CCW)** and reverses to **M03 (CW)**

Applications: - Left-hand threads - Reverse boring - Counter-boring operations

G84.2/G84.3 - Tapping with Peck (Control-Dependent)

Some controls support peck tapping for chip breaking:

G84.2 X__ Y__ Z__ R__ Q__ F__

Sequence: - Feed-tap Q distance - Reverse spindle briefly - Forward spindle - Continue until depth

Not universally supported - check control manual.

Boring Cycles

G85 - Boring Cycle (Feed In, Feed Out)

Simple boring with feed in and feed out:

G85 X__ Y__ Z__ R__ F__

Sequence: 1. Rapid to XY at current Z 2. Rapid to R plane 3. Feed to Z depth 4. **Feed out to R plane** (no rapid)

Applications: - Precision holes requiring good finish on exit - Boring bars sensitive to rapid retract - When chip evacuation during retract is needed

Example:

G99 G85 X50 Y50 Z-25 R5.0 F80 (Bore hole, feed in and out)

G86 - Boring Cycle (Feed In, Spindle Stop, Rapid Out)

Boring with spindle stop at depth before rapid retract:

G86 X__ Y__ Z__ R__ F__

Sequence: 1. Rapid to XY at current Z 2. Rapid to R plane 3. Feed to Z depth 4. **Spindle stop (M05)** 5. Rapid out to R plane 6. **Spindle start (M03)**

Applications: - Avoiding tool marks on hole finish - Boring bars that deflect under cutting forces - Precision bore diameter control

Limitation: Slower due to spindle stop/start

G87 - Back Boring Cycle

Advanced boring cycle for interrupted cuts (not universally supported):

G87 X__ Y__ Z__ R__ Q__ F__

Sequence: 1. Orient spindle (M19) 2. Rapid away from hole 3. Rapid to depth 4. Feed bore 5. Retract

Rare in modern practice - primarily for horizontal boring mills.

G88 - Boring Cycle (Feed In, Spindle Stop, Manual Retract)

Boring with manual retract:

G88 X__ Y__ Z__ R__ P__ F__

Sequence: 1. Feed to depth 2. Dwell P seconds 3. Spindle stop 4. **Wait for manual retract** (operator jogs Z)

Application: Very large or delicate bores where automatic retract may cause damage.

G89 - Boring Cycle (Feed In, Dwell, Feed Out)

Precision boring with dwell at depth:

G89 X__ Y__ Z__ R__ P__ F__

Sequence: 1. Rapid to XY at current Z 2. Rapid to R plane 3. Feed to Z depth 4. **Dwell P seconds** 5. Feed out to R plane

Applications: - Precision hole sizing - Excellent surface finish - Relieving cutting forces before retract

Example:

G99 G89 X50 Y50 Z-30 R5.0 P2.0 F50 (Precision bore with 2-sec dwell)

Canned Cycle Summary Table

Cycle	Name	Feed In	Action at Depth	Retract	Application
G80	Cancel	-	-	-	Cancel active cycle
G81	Drill	Yes	None	Rapid	Simple drilling
G82	Drill/Dwell	Yes	Dwell	Rapid	Chip break, finish
G83	Peck Drill	Yes	Full retract each peck	Rapid	Deep holes
G73	Hi-Speed Peck	Yes	Partial retract	Rapid	Cast iron, short chips
G84	Tap	Yes	Spindle reverse	Feed out	Right-hand threads
G74	Left Tap	Yes	Spindle reverse	Feed out	Left-hand threads
G85	Bore	Yes	None	Feed	Precision boring
G86	Bore	Yes	Spindle stop	Rapid	Fine finish, no marks
G89	Bore/Dwell	Yes	Dwell	Feed	Ultra-precision

Practical Examples

Example 1: Bolt Circle (4 Holes)

G21 G90 G17 G54

T01 M06 (6mm drill)

G43 H01

S2000 M03

M08

G00 X50 Y50 Z10.0 (Center of 100mm bolt circle)

(Drill 4 holes on 100mm BC)

G99 G83 Z-15 R5.0 Q5.0 F100 (Peck drill)

X100 Y50 (0 deg)

```

X50 Y100          (90 deg)
X0 Y50            (180 deg)
X50 Y0            (270 deg)
G80              (Cancel)

```

```

G00 Z50.0
M09
M05
M30

```

Example 2: Drilling, Tapping Sequence

```
G21 G90 G17 G54
```

```

(--- TOOL 1: 5MM DRILL (TAP DRILL FOR M6x1.0) ---)
T01 M06
G43 H01
S1500 M03
M08

```

```

G99 G81 Z-18 R5.0 F120          (Drill tap holes)
X10 Y10
X40 Y10
X10 Y40
X40 Y40
G80

```

```

M09
M05
G00 Z50.0

```

```

(--- TOOL 2: M6x1.0 TAP ---)
T02 M06
G43 H02
S400 M03          (400 RPM for tapping)

```

```

G99 G84 Z-15 R5.0 F400          (F = 400 RPM x 1.0 pitch)
X10 Y10
X40 Y10
X10 Y40
X40 Y40
G80

```

```

M05
M30

```

Example 3: Mixed Hole Depths

```

G21 G90 G17 G54
T01 M06
G43 H01
S2000 M03
M08

```

```
(Shallow holes)
```

```
G98 G81 Z-10 R5.0 F100
X10 Y10
X20 Y10
G80
```

```
(Deep holes - change to peck cycle)
G98 G83 Z-50 R5.0 Q8.0 F80
X30 Y10
X40 Y10
G80
```

```
(Counterbore - use G82 with dwell)
G98 G82 Z-8 R5.0 P1.0 F60
X10 Y10 (Counterbore first hole)
X20 Y10 (Counterbore second hole)
G80
```

```
M09
M05
M30
```

Example 4: Pattern with Subprogram

```
(Main program - 3 rows of holes)
O1000
G21 G90 G17 G54
T01 M06
G43 H01
S2000 M03
M08
```

```
G00 Y0 (Row 1)
M98 P2000 (Call hole pattern)
G00 Y25 (Row 2)
M98 P2000
G00 Y50 (Row 3)
M98 P2000
```

```
M09
M05
M30
```

```
(Subprogram - row of 5 holes)
O2000
G99 G81 Z-15 R5.0 F100
X0
X25
X50
X75
X100
G80
M99
```

Troubleshooting Canned Cycles

Common Errors

1. Feed rate not set:

G81 X10 Y10 Z-20 R5.0 (ERROR: No F-word)

Solution: Always specify F in first cycle call.

2. R plane below surface:

G81 X10 Y10 Z-20 R-5.0 F100 (R below Z depth - invalid)

Solution: R plane must be above Z depth ($R > Z$ in absolute mode).

3. Wrong return mode:

G98 G81 X10 Y10 Z-20 R5.0 F100 (Returns to high Z unnecessarily)

Solution: Use G99 for closely-spaced holes.

4. Tapping feed rate incorrect:

S500 M03

G84 X10 Y10 Z-20 R5.0 F100 (Wrong: Should be F500 for 1mm pitch)

Solution: $F = S \times \text{Pitch}$

Verification Techniques

1. **Simulate program** before running
2. **Single block mode** for first hole
3. **Dry run** above work surface
4. **Check R plane clearance** with tool at $Z=R$
5. **Verify feed rate** calculation for tapping

Key Takeaways

1. **Canned cycles** simplify repetitive operations into single-block commands
2. **G81-G89** cover drilling, tapping, and boring operations
3. **G98/G99** control return height (initial Z or R plane)
4. **G83** (peck drilling) essential for deep holes and chip management
5. **G84** (tapping) requires **$F = S \times \text{Pitch}$** calculation
6. **Q parameter** controls peck increment
7. **P parameter** specifies dwell time
8. **G80** cancels active canned cycle
9. **Modal behavior** repeats cycle at each new XY position
10. **Proper R plane selection** prevents crashes and optimizes cycle time

Next: Section 15.7 - Programming Best Practices

Previous: Section 15.5 - Auxiliary Functions

Section 15.7 - Programming Best Practices

Overview

Professional G-code programming requires more than syntactic correctness—it demands defensive coding, clear documentation, efficient toolpath design, and maintainable structure. Best practices minimize errors, reduce cycle time, simplify troubleshooting, and ensure programs remain usable years after creation.

This section presents industry-proven techniques for writing robust, efficient, and maintainable CNC programs.

Program Structure and Organization

Standard Program Template

A well-structured program follows consistent organization:

```
%
01234 (PROGRAM NUMBER)
(=====)
(PART: BRACKET-100-REV-C)
(MATERIAL: 6061-T6 ALUMINUM)
(PROGRAMMER: J. SMITH)
(DATE: 2025-01-15)
(SETUP: VISE JAW 1, SOFT JAWS)
(ORIGIN: CENTER-CENTER-TOP OF PART)
(STOCK: 125 X 75 X 25MM)
(=====)

(TOOL LIST)
(T01: 12MM 4-FL CARBIDE END MILL - ROUGH)
(T02: 6MM 4-FL CARBIDE END MILL - FINISH)
(T03: 8MM CARBIDE DRILL)
(T04: M10X1.5 SPIRAL TAP)

(=====)
(INITIALIZATION)
(=====)
G21 G90 G17 G94      (Metric, absolute, XY plane, feed/min)
G40 G49 G80          (Cancel comp, cancel offsets, cancel cycles)
G54                  (Work offset 1)

(=====)
(TOOL 1: 12MM END MILL - ROUGHING)
(=====)
T01 M06
G43 H01 Z50.0
S2000 M03
G04 P2.0
M08

(Roughing operations...)
G00 X0 Y0
G00 Z5.0
G01 Z-5.0 F100
G01 X50.0 F500
```

```

(...more operations...)

G00 Z50.0
M09
M05

(=====)
(TOOL 2: 6MM END MILL - FINISHING)
(=====)
T02 M06
G43 H02 Z50.0
S3000 M03
G04 P2.0
M08

(Finishing operations...)
(...operations...)

G00 Z50.0
M09
M05

(=====)
(CLEANUP AND END)
(=====)
G00 Z50.0
G28 G91 Z0          (Home Z-axis)
G28 X0 Y0          (Home XY-axes)
G90                (Restore absolute mode)
M30                (End program, rewind)
%
```

Header Documentation

Essential header information:

(PART: BRACKET-100-REV-C)	(Part number with revision)
(MATERIAL: 6061-T6 ALUMINUM)	(Material specification)
(PROGRAMMER: J. SMITH)	(Accountability)
(DATE: 2025-01-15)	(Creation/modification date)
(SETUP: VISE JAW 1, SOFT JAWS)	(Fixturing details)
(ORIGIN: CENTER-CENTER-TOP)	(Work offset location)
(STOCK: 125 X 75 X 25MM)	(Raw material size)
(NOTES: DEBUR ALL EDGES AFTER)	(Special instructions)

Tool list documentation:

```

(TOOL LIST)
(T01: 12MM 4-FL CARBIDE END MILL)
( PURPOSE: ROUGHING)
( RPM: 2000 FEED: 500MM/MIN)
( LENGTH: 150.325MM)
( NOTES: CHECK FOR WEAR EVERY 5 PARTS)

(T02: 6MM DRILL)
( PURPOSE: PILOT HOLES FOR M8 TAP)
```

(RPM: 3000 FEED: 150MM/MIN)

Section Dividers

Use visual dividers to separate operations:

```
(=====)
(OPERATION 10: FACE TOP SURFACE)
(=====)
```

or

```
(=====)
(* POCKET ROUGHING - 12MM END MILL *)
(=====)
```

Safety and Initialization

Safety Block

Always initialize modal states at program start:

```
(SAFETY/INITIALIZATION BLOCK)
G21                (Metric units - EXPLICIT)
G90                (Absolute positioning)
G17                (XY plane selection)
G40                (Cancel cutter comp)
G49                (Cancel tool length offset)
G80                (Cancel canned cycles)
G54                (Work offset 1)
G94                (Feed per minute)
G64 P0.01          (Path blending with tolerance)
```

Why explicit initialization? - Unknown modal state from previous program - Operator may have changed settings - Power interruption may have reset controller - Different operator habits

Safe Tool Change Sequence

Standard tool change protocol:

```
(SAFE TOOL CHANGE)
G00 Z50.0          (Retract to safe Z - FIRST)
M09                (Coolant off)
M05                (Spindle off)
G04 P3.0           (Wait for spindle stop)
G28 G91 Z0         (Home Z-axis - optional but recommended)
G90                (Restore absolute mode)
T02 M06            (Now safe to change tool)
G43 H02 Z50.0      (Apply new tool offset, position at safe Z)
```

Critical sequence: 1. **Retract Z first** (prevents table crash during XY move) 2. **Stop coolant and spindle** 3. **Wait for spindle to stop completely** 4. **Execute tool change** 5. **Apply new tool offset immediately**

Soft Limit Checking

Add explicit checks for out-of-range moves:

```

(Check machine travel limits)
#1 = 500.0           (Machine X+ limit)
#2 = 200.0           (Programmed X move)

O100 IF [#2 GT #1]
  (ABORT, MOVE EXCEEDS X+ LIMIT)
O100 ENDIF

```

Efficient Toolpath Design

Minimize Air Cutting

Poor practice:

```

G00 Z50.0           (Retract to 50mm)
G00 X100.0 Y50.0    (Rapid to next position)
G00 Z5.0            (Rapid down)
G01 Z-10.0 F100     (Feed to depth)

```

Better practice:

```

G00 Z10.0           (Retract only to clearance height)
G00 X100.0 Y50.0    (Rapid to next position at lower Z)
G00 Z2.0            (Rapid to near cutting depth)
G01 Z-10.0 F100     (Short feed to depth)

```

Savings: Reduced Z-axis travel = faster cycle time, less wear.

Optimize Rapid Order

Poor practice:

```

G00 X100.0 Y50.0    (Long diagonal rapid with tool extended)
G00 Z5.0            (Then retract)

```

Better practice:

```

G00 Z5.0            (Retract first)
G00 X100.0 Y50.0    (Then rapid - safer)

```

Use G64 for Continuous Contouring

For roughing (speed priority):

```

G64 P0.05           (Blend paths within 0.05mm)
G01 X10 Y10 F500
G01 X20 Y10
G01 X20 Y20         (Corners rounded for speed)

```

For finishing (accuracy priority):

```

G64 P0.005          (Tighter tolerance, 0.005mm)
G01 X10 Y10 F200
G01 X20 Y10
G01 X20 Y20         (Sharper corners)

```

For critical dimensions:

```

G61                 (Exact stop mode)
G01 X10 Y10 F200
G01 X20 Y10         (Full stop at corner)

```

Reduce Tool Changes

Group operations by tool rather than by feature:

Poor practice:

```
(Tool 1: Drill hole 1)
(Tool 2: Tap hole 1)
(Tool 1: Drill hole 2)
(Tool 2: Tap hole 2)
(4 tool changes total)
```

Better practice:

```
(Tool 1: Drill all holes)
(Tool 2: Tap all holes)
(2 tool changes total)
```

Feed Rate and Speed Optimization

Adaptive Feed Rates

Vary feed rates based on operation:

```
(ENTRY MOVE - REDUCED FEED)
G01 Z-10.0 F50          (Slow plunge to avoid shock load)

(CUTTING MOVE - FULL FEED)
G01 X50.0 F500          (Full feed for side cutting)

(EXIT MOVE - REDUCED FEED)
G01 Z5.0 F100           (Moderate feed for retraction)
```

Step-Down Strategy

For deep pockets, use multiple passes:

```
(POCKET ROUGHING - 3 PASSES AT -5MM EACH)
G01 Z-5.0 F100          (Pass 1: Top 5mm)
(...pocket path...)
G01 Z-10.0 F100         (Pass 2: Next 5mm)
(...pocket path...)
G01 Z-15.0 F100         (Pass 3: Final 5mm)
(...pocket path...)
```

Depth per pass calculation:

$$D_{pass} = \frac{D_{tool} \times 0.3 \text{ to } 0.5}{\text{hardness factor}}$$

Climb vs. Conventional Milling

Climb milling (preferred for CNC):

```
G01 X50.0 Y0 F500       (Tool rotation direction matches feed direction)
G01 X50.0 Y25.0         (Cutting moves left-to-right on right edge)
```

Advantages: - Better surface finish - Less tool wear - Lower cutting forces - Chips evacuate behind tool

Requirements: - Rigid machine with minimal backlash - Sharp tooling - Adequate workholding

Conventional milling (when needed): - Soft materials prone to work hardening (stainless) - Very worn tooling - Machines with backlash issues

Error Prevention

Defensive Programming Techniques

1. Check for zero feed rate:

```
O100 IF [#_feed EQ 0]
  (ABORT, FEED RATE IS ZERO)
O100 ENDIF
G01 X10.0 Y10.0
```

2. Validate tool offsets:

```
O200 IF [#5403 EQ 0]    (Check if H offset is set)
  (ABORT, TOOL LENGTH OFFSET NOT SET)
O200 ENDIF
```

3. Pre-position before cycle:

```
G00 Z50.0              (Always at known Z before work offset change)
G55                    (Safe to change offset)
```

Common Mistakes to Avoid

1. G90/G91 confusion:

```
(WRONG - Mixed modes without awareness)
G90 G01 X10.0          (Absolute)
G91 G01 X10.0          (Incremental - now at X20!)
G01 X10.0              (Still incremental - now at X30!)
```

```
(CORRECT - Explicit mode setting)
G90
G01 X10.0
G01 X20.0
```

2. Feed rate not modal across tool changes:

```
(WRONG)
T01 M06
G01 X10.0 F500          (Feed set for T01)
T02 M06                 (Feed still 500)
G01 X20.0               (May be too fast for T02)
```

```
(CORRECT)
T01 M06
G01 X10.0 F500
T02 M06
G01 X20.0 F200          (Explicit feed for T02)
```

3. Forgetting to cancel cycles:

```
(WRONG)
G81 X10 Y10 Z-20 R5 F100
X20 Y10
G00 X50 Y50             (G81 still active - drills here!)
```

```
(CORRECT)
G81 X10 Y10 Z-20 R5 F100
X20 Y10
G80                (Cancel cycle)
G00 X50 Y50
```

4. Arc radius tolerance:

```
(WRONG - Radius mismatch)
G00 X0 Y0
G02 X10 Y10 I10 J0    (Start radius 10, end radius 7.07 - ERROR)
```

```
(CORRECT - Verify geometry)
G00 X0 Y0
G02 X10 Y0 I5 J0      (90 deg arc, consistent 5mm radius)
```

Code Clarity and Maintainability

Meaningful Comments

Poor comments (redundant):

```
G01 X10.0          (Move to X10)
G01 Y20.0          (Move to Y20)
```

Better comments (explain intent):

```
G01 X10.0          (Align with left edge of pocket)
G01 Y20.0          (Position for corner radius entry)
```

Best comments (document assumptions and gotchas):

```
G01 Z-10.5 F100     (Depth adjusted +0.5mm for spring-back)
G04 P2.0            (Dwell required for aluminum chip clearing)
G41 D01             (Comp left - CHECK: Must be outside profile)
```

Use Subprograms for Repetition

Without subprograms:

```
(Pocket pattern 1)
G01 X10 Y10
G01 X20 Y10
(...50 lines...)

(Pocket pattern 2 - same shape, different position)
G01 X40 Y10
G01 X50 Y10
(...50 lines duplicated...)
```

With subprograms:

```
(Main program)
G00 X10 Y10          (Position for pocket 1)
M98 P5000            (Call pocket subprogram)
G00 X40 Y10          (Position for pocket 2)
M98 P5000            (Call pocket subprogram)
```

```
(Subprogram 05000 - pocket pattern)
05000
```

```

G91                      (Incremental mode for reusable pattern)
G01 X10 Y0 F500
G01 Y10
(...pattern continues...)
G90                      (Restore absolute)
M99

```

Parametric Programming for Families

Use variables for part families:

```

(PARAMETRIC BOLT CIRCLE)
#1 = 100.0                (Bolt circle diameter)
#2 = 8                    (Number of holes)
#3 = 360.0 / #2           (Angle between holes)

O100 D0 [#4 = 1, #2]      (Loop for each hole)
  #5 = [#1/2] * COS[#3 * #4] (X position)
  #6 = [#1/2] * SIN[#3 * #4] (Y position)
  G81 X#5 Y#6 Z-20 R5 F100   (Drill hole)
O100 ENDDO

G80

```

Version Control and Documentation

Revision History

Include change log in program header:

```

(REVISION HISTORY)
(REV-A: 2025-01-15 - INITIAL RELEASE - J.SMITH)
(REV-B: 2025-01-20 - INCREASED FEED RATES 10% - J.SMITH)
(REV-C: 2025-02-05 - ADDED TAP CYCLE FOR M8 HOLES - A.JONES)
(CURRENT REV: C)

```

External Documentation

Maintain external documentation: - **Setup sheets:** Fixtures, workholding, tool list - **Inspection reports:** First article, in-process checks - **Tool life records:** Hours, parts count, replacement schedule - **Material certifications:** Heat lot, mechanical properties

Backup and Archiving

Best practices: - Version control system (Git, SVN) for program files - Regular backups to network storage - Archive proven programs with revision notes - Document tool offsets and work offset values - Photograph setups for future reference

Testing and Validation

Simulation Before First Run

Always simulate:

```

(Verify in CAM software or standalone simulator)
1. Check all motions for collisions
2. Verify tool reaches all features
3. Confirm feed rates are reasonable

```

4. Check spindle speed limits
5. Validate work offset and tool offset logic

Dry Run Procedure

Step 1: Dry run with offsets:

(Run program 25mm above work surface)

- Load all tools
- Set work offset Z = +25.0 (instead of 0)
- Run program at 100% feed
- Watch for unusual motion, verify cycle time

Step 2: Dry run with tool measurement:

(Run with actual offsets but no spindle/coolant)

- Set correct work offsets
- Disable spindle start (manual override or M03/M08 removal)
- Run in single block mode
- Verify tool doesn't crash

Step 3: First article run:

(Run actual program with reduced feed)

- Set feed override to 50%
- Run in single block for first few moves
- Increase override gradually
- Inspect first part carefully

Single Block Execution

Run critical sections line-by-line:

(Use single-block mode for:)

- First tool approach to workpiece
- Tool changes
- Work offset changes
- Complex contouring near fixtures
- Final finishing pass

Program Optimization Checklist

Before finalizing a program:

- ☐ All modal states initialized at start
- ☐ Tool change sequence includes spindle stop, coolant off
- ☐ Feed rates appropriate for tool and material
- ☐ Spindle speeds within tool and machine limits
- ☐ Canned cycles canceled before non-cycle moves
- ☐ Work offsets and tool offsets documented
- ☐ Comments explain non-obvious operations
- ☐ Repetitive operations use subprograms or loops
- ☐ Air cutting minimized (rapids at appropriate heights)
- ☐ Tool changes grouped to minimize count
- ☐ Program simulated and verified
- ☐ Revision history and header complete

Key Takeaways

1. **Structure programs** with clear headers, sections, and comments
2. **Initialize all modal states** at program start (G21, G90, G17, G40, G49, G80)
3. **Safe tool change sequence:** Retract Z, stop coolant/spindle, wait, change tool
4. **Optimize toolpaths:** Minimize air cutting, group by tool, use path blending
5. **Defensive programming:** Check for errors, validate inputs, explicit mode setting
6. **Clear comments** explain intent and assumptions, not obvious syntax
7. **Use subprograms** for repetitive patterns and part families
8. **Test thoroughly:** Simulate, dry run, single block execution
9. **Document everything:** Tool list, setup, revisions, assumptions
10. **Maintain code quality** for long-term usability and troubleshooting

Next: Section 15.8 - Post-Processing

Previous: Section 15.6 - Canned Cycles

Section 15.8 - Post-Processing and CAM Integration

Overview

Post-processors are the critical translation layer between CAM software and CNC machines, converting generic toolpath data into machine-specific G-code dialects. Understanding post-processor architecture, configuration, and customization enables optimization of CAM output for specific machines, control systems, and operational requirements.

This section covers post-processor fundamentals, common CAM systems, configuration techniques, and strategies for debugging and customizing post-processors.

The CAM to G-Code Workflow

Complete Toolpath Generation Pipeline

```
CAD Model
  down
CAM Toolpath Calculation (Generic)
  down
Post-Processor (Machine-Specific)
  down
G-Code Program
  down
CNC Controller
  down
Physical Part
```

What the Post-Processor Does

Input: Generic toolpath data (APT, CL data) - Tool positions (X, Y, Z coordinates) - Tool identifiers
- Spindle speeds - Feed rates - Coolant on/off - Tool change commands

Output: Machine-specific G-code - Control-specific syntax (FANUC, Siemens, Heidenhain) - Machine kinematics compensation - Tool library format - Canned cycle preferences - Safe positioning sequences - Custom M-codes

Transformations: 1. **Syntax translation:** Generic commands -> G-code dialect 2. **Kinematic correction:** Linear coordinates -> rotary axis angles (5-axis) 3. **Tool compensation:** Apply or cancel based on control capability 4. **Cycle optimization:** Convert linear moves to canned cycles 5. **Safety insertion:** Add initialization blocks, tool change sequences 6. **Formatting:** Line numbers, decimal places, comments

Post-Processor Architecture

Generic Post-Processor Structure

Most post-processors follow this organization:

```
POST-PROCESSOR FILE (.cps, .pst, .scl, .def)
|-- Machine Definition
|   |-- Axis configuration (3/4/5-axis)
|   |-- Travel limits
|   |-- Rotary axis directions
|   +-- Kinematics model
|-- Control Definition
|   |-- G-code dialect (FANUC, Siemens, etc.)
|   |-- Supported commands
|   |-- Modal group rules
```

```

|   +-- Output format
|-- Formatting Rules
|   |-- Decimal places
|   |-- Line numbering
|   |-- Comment style
|   +-- Block structure
|-- Output Sections
|   |-- Program header
|   |-- Tool change sequence
|   |-- Motion commands
|   |-- Cycle definitions
|   +-- Program footer
+-- Custom Functions
    |-- Special operations
    |-- Macro calls
    +-- User-defined logic

```

Post-Processor Languages

Different CAM systems use different post-processor formats:

CAM System	Post Language	File Extension
Fusion 360	JavaScript	.cps
Mastercam	Post Descriptor Language	.pst
SolidCAM	SPL (SolidPost Language)	.scl
NX CAM	Template Control Language (TCL)	.tcl
Edgecam	Post Generator Macro	.pmx
ESPRIT	Post Definition	.def

Common Post-Processor Settings

Machine Configuration

Axis configuration:

```

// Fusion 360 CPS example
var machineConfiguration = new MachineConfiguration();
machineConfiguration.setNumberOfAxes(3); // 3-axis mill
machineConfiguration.setModel("Haas VF-2");
machineConfiguration.setVendor("Haas");

```

Travel limits:

```

xAxisMinimum = -508; // mm
xAxisMaximum = 508;
yAxisMinimum = -406;
yAxisMaximum = 406;
zAxisMinimum = -508;
zAxisMaximum = 0;

```

Rotary axis setup (4th/5th axis):

```

var aAxis = createAxis({coordinate:0, table:true, axis:[1,0,0], range:[-120,120], preference:1});
var cAxis = createAxis({coordinate:2, table:true, axis:[0,0,1], range:[-360,360], cyclic:true});

```

Control Dialect Selection

FANUC-style:

```
properties.controllerType = "fanuc";
properties.useG28 = true;           // G28 for homing
properties.useM06 = true;          // M06 for tool change
properties.sequenceNumberStart = 10; // Line numbering
properties.sequenceNumberIncrement = 10;
```

Siemens-style:

```
properties.controllerType = "siemens";
properties.useG28 = false;        // Siemens uses different homing
properties.useM06 = true;
properties.useCycles = true;      // Use Siemens drilling cycles
```

LinuxCNC:

```
properties.controllerType = "linuxcnc";
properties.useToolChanger = false;    // Manual tool changes
properties.useG43 = true;             // Tool length compensation
properties.separateWordsWithSpace = true;
```

Output Formatting

Decimal precision:

```
var xyzFormat = createFormat({decimals:3, forceDecimal:true}); // 0.001mm
var feedFormat = createFormat({decimals:0});                  // Integer feed
var rpmFormat = createFormat({decimals:0});                  // Integer RPM
```

Line numbering:

```
properties.showSequenceNumbers = true;
properties.sequenceNumberStart = 10;
properties.sequenceNumberIncrement = 5;
properties.sequenceNumberOnlyOnToolChange = false;
```

Comment style:

```
properties.useParentheses = true;    // (Comment style)
// vs.
properties.useSemicolon = true;      // ; Comment style
```

Tool Change Behavior

Automatic tool changer:

```
function onToolChange() {
    writeBlock("M05");           // Spindle stop
    writeBlock("M09");           // Coolant off
    writeBlock("G28 G91 Z0");     // Home Z
    writeBlock("G28 X0 Y0");     // Home XY (optional)
    writeBlock("G90");           // Restore absolute
    writeBlock("T" + toolFormat.format(tool.number), "M06"); // Tool change
    writeBlock("G43", "H" + toolFormat.format(tool.number)); // Tool offset
}
```

Manual tool change:

```

function onToolChange() {
    writeBlock("M05");
    writeBlock("M09");
    writeBlock("G28 G91 Z0");
    writeBlock("M00"); // Program stop for manual change
    writeln("(LOAD TOOL " + tool.number + " - " + tool.description + ")");
    writeBlock("G43", "H" + toolFormat.format(tool.number));
}

```

Customizing Post-Processors

Common Customizations

1. Add custom M-codes for specific equipment:

```

// Add M-code for automatic part probe
function onProbeStart() {
    if (properties.useProbe) {
        writeBlock("M75"); // Enable probe
    }
}

function onProbeEnd() {
    if (properties.useProbe) {
        writeBlock("M76"); // Disable probe
    }
}

```

2. Insert dwell after spindle start:

```

function onSpindleStart() {
    writeBlock("S" + rpmFormat.format(spindleSpeed), "M03");
    writeBlock("G04 P" + (properties.spindleWaitTime * 1000)); // Dwell in ms
}

```

3. Customize program header:

```

function onProgramHeader() {
    writeln("%");
    writeln("O" + oFormat.format(programNumber));
    writeln("(PROGRAM: " + programName + ")");
    writeln("(DATE: " + new Date().toISOString().split('T')[0] + ")");
    writeln("(MATERIAL: " + getGlobalParameter("part-material") + ")");
    writeln("(OPERATOR: CHECK TOOL OFFSETS BEFORE RUNNING)");
    writeln("(=====)");
}

```

4. Add tool list to header:

```

function writeToolList() {
    writeln("(TOOL LIST)");
    var tools = getToolTable();
    for (var i = 0; i < tools.getNumberOfTools(); i++) {
        var tool = tools.getTool(i);
        writeln("(T" + toolFormat.format(tool.number) + ": " +
            tool.description + " - DIA:" +
            xyzFormat.format(tool.diameter) + ")");
    }
}

```

```
writeln("(=====)");
}
```

5. Implement custom safe retract:

```
function onSafeRetract() {
  if (properties.useG28) {
    writeBlock("G28 G91 Z0");
    writeBlock("G28 X0 Y0");
    writeBlock("G90");
  } else {
    writeBlock("G53 G00 Z0"); // Machine coordinate Z home
  }
}
```

Adding Post-Processor Properties

Properties allow users to configure post-processor behavior without editing code:

```
// Define properties
properties = {
  useToolChanger: true,
  spindleWaitTime: 2.0,
  useCoolant: true,
  optimizeRapids: false,
  safeRetractHeight: 50.0,
  minimumChordLength: 0.01,
  minimumCircularRadius: 0.01,
  maximumCircularRadius: 1000.0,
  allowHelicalMoves: true,
  useG28ForToolChange: true,
  probeOnToolChange: false
};

// Use properties in code
if (properties.useToolChanger) {
  writeBlock("T" + toolFormat.format(tool.number), "M06");
} else {
  writeBlock("M00");
  writeln("(MANUALLY CHANGE TO TOOL " + tool.number + ")");
}
```

Debugging Post-Processor Output

Common Post-Processor Issues

1. Missing or incorrect initialization:

Problem:

(No G21/G20, G90/G91, or work offset specified)
T01 M06
G00 X10 Y10

Solution: Ensure post writes initialization block:

```
function writeInitialization() {
  writeBlock("G21"); // Metric
```

```

writeBlock("G90"); // Absolute
writeBlock("G17"); // XY plane
writeBlock("G54"); // Work offset 1
writeBlock("G40 G49 G80"); // Cancel compensations
}

```

2. Incorrect arc output:

Problem:

G02 X10 Y10 I5 J5 (Arc center calculation wrong)

Solution: Check arc output format in post:

```

if (isHelical()) {
  writeBlock("G02",
    x, y, z,
    "I" + xyzFormat.format(cx - start.x),
    "J" + xyzFormat.format(cy - start.y),
    feed);
}

```

3. Tool length offset not applied:

Problem:

T01 M06
 (Missing G43 H01)
 G00 Z0 (Crash - no offset active!)

Solution: Add G43 after every tool change:

```

function onToolChange() {
  writeBlock("T" + toolFormat.format(tool.number), "M06");
  writeBlock("G43", "H" + toolFormat.format(tool.number)); // Critical!
}

```

4. Feed rate not output:

Problem:

G01 X10 Y10 (F-word missing)

Solution: Force feed output on mode change:

```

var feedOutput = createVariable({force:true}, feedFormat);
writeBlock("G01", x, y, feedOutput.format(feed));

```

Post-Processor Testing Workflow

Step 1: Simple test part - Single tool - Basic rectangle - One depth - No complex features

Step 2: Verify output

(Check for:)

- Program start (%/Oxxxx)
- Initialization (G21/G90/G17/G54)
- Tool change sequence
- First rapid position
- First cut with feed rate
- Program end (M30)

Step 3: Incremental complexity - Multiple tools - Drilling cycles - Circular interpolation - Helical moves
- Different planes (G17/G18/G19)

Step 4: Simulation - Load G-code into CAMotics, NC Viewer, or machine simulator - Check for crashes, unexpected motion - Verify tool paths match CAM preview

CAM System-Specific Guidance

Fusion 360 Post-Processors

Location:

C:/Users/<username>/AppData/Local/Autodesk/Autodesk Fusion 360/CAM/cache/posts

Editing: - Posts are JavaScript (.cps files) - Edit with text editor (VS Code recommended) - Reload in Fusion after changes (close and reopen CAM)

Common modifications:

```
// Change decimal places
var xyzFormat = createFormat({decimals:4});

// Add custom property
properties.customProperty = "default value";

// Modify tool change
function onToolChange() {
    // Custom tool change sequence
}
```

Mastercam Post-Processors

Location:

C:/Users/Public/Documents/shared Mcam2024/mill/Posts

Editing: - Posts use .pst format (text-based) - Edit with Mastercam Post Editor or text editor - Complex syntax, steeper learning curve

Structure:

```
# Header information
# Machine definition
# Formatting
# Tool change logic
# Output blocks
```

SolidCAM Post-Processors

Post Generator: - Graphical post configuration tool - Generates .scl file - Less direct code editing, more UI-driven

Advantages: - Easier for beginners - Consistent structure - Built-in validation

Disadvantages: - Less flexibility for advanced customization - Some operations require scripting knowledge

Best Practices for Post-Processing

1. Start with Manufacturer Post

Most controls have vendor-provided posts: - Haas (Haas mill/lathe posts) - FANUC (generic FANUC posts) - Siemens (generic Siemens posts) - LinuxCNC (generic LinuxCNC posts)

Advantages: - Pre-configured for control dialect - Tested on actual machines - Includes quirks and workarounds

2. Document All Modifications

```
// Modified 2025-01-15 by J.Smith  
// Added 2-second dwell after spindle start for high-speed spindle  
function onSpindleStart() {  
    writeBlock("S" + rpmFormat.format(spindleSpeed), "M03");  
    writeBlock("G04 P2.0"); // ADDED: Spindle stabilization dwell  
}
```

3. Version Control Posts

- Keep original post as backup (.cps.original)
- Use version numbers in custom posts (haas-vf2-custom-v1.2.cps)
- Document changes in comments
- Test thoroughly before production use

4. Validate with Multiple Part Types

Test post with: - Simple 2D contours - 3D surfaces - Drilling operations - Threading/tapping - Multiple work offsets - Tool changes

5. Coordinate with Operators

- Document post-specific requirements
- Note any manual steps (tool measurement, probe setup)
- Include setup sheets with programs
- Provide examples of expected output

Advanced Post-Processing Topics

5-Axis Kinematics

5-axis machines require complex kinematics compensation:

```
// Rotary axis limits  
var aAxisMinimum = -120;  
var aAxisMaximum = 120;  
var cAxisMinimum = -360;  
var cAxisMaximum = 360;  
  
// Tool center point management  
function onCircular5D() {  
    linearize(tolerance); // Convert to linear moves if too complex  
}
```

Multi-Axis Synchronization

Mill-turn machines require coordinated main/sub spindle:

```
function onSubSpindleTransfer() {
  writeBlock("M154");      // Sub-spindle grip
  writeBlock("G28 U0");    // Retract main spindle
  writeBlock("M155");      // Transfer to sub-spindle
}
```

Adaptive Toolpath Support

Modern CAM systems generate variable-feed toolpaths:

```
function onFeedRateChange() {
  if (properties.adaptiveFeed) {
    forceModals();
    feedOutput.reset();
    writeBlock("G01", feedOutput.format(feed));
  }
}
```

Post-Processor Resources

Documentation

- **Fusion 360:** Help -> CAM -> Posts (built-in documentation)
- **Mastercam:** Post Processor Reference Guide (PDF)
- **SolidCAM:** Post Generator User Guide
- **CAMplete:** TruePath Post documentation

Community Resources

- **Autodesk Forums:** Fusion 360 CAM and Post-Processors section
- **Mastercam Forum:** Post Processor Development
- **Practical Machinist:** CAM software subforum
- **CNCZone:** CAM Software section

Sample Posts

- **Fusion 360 post library:** Built-in, 200+ posts
- **GitHub:** Search for “fusion360-post” or “mastercam-post”
- **Machine vendor websites:** Often provide tested posts

Key Takeaways

1. **Post-processors** translate generic CAM toolpaths to machine-specific G-code
2. **Start with vendor-provided posts** and customize incrementally
3. **Document all modifications** with comments and version control
4. **Test thoroughly** with simulation before running on machine
5. **Common customizations:** Tool change sequence, initialization, custom M-codes, header format
6. **Debugging:** Check initialization, feed rates, tool offsets, arc output
7. **Properties** allow user configuration without code editing
8. **Different CAM systems** use different post languages (JavaScript, PST, SCL, TCL)
9. **Validate with multiple part types** to ensure robustness
10. **Coordinate with operators** to document post-specific requirements

Next: Section 15.9 - Advanced Features

Previous: Section 15.7 - Programming Best Practices

Section 15.9 - Advanced Features: Macros, Variables, and Parametric Programming

Overview

Advanced G-code features—variables, expressions, conditional logic, loops, and subprograms—transform static programs into flexible, parametric systems. These capabilities enable adaptive machining, part families, automatic error recovery, and in-process measurement integration.

This section covers parametric programming techniques, macro variables, control flow, mathematical expressions, and practical applications for advanced CNC programming.

Variables and Parameters

System Variables (Read-Only)

System variables provide access to machine state and position:

Variable	Description	Example Value
#5161-#5166	G28 home position (X, Y, Z, A, B, C)	0.0
#5181-#5186	G30 secondary home (X, Y, Z, A, B, C)	100.0
#5201-#5206	G54 work offset (X, Y, Z, A, B, C)	-200.0
#5221-#5226	G55 work offset	-180.5
#5241-#5326	G56-G59 work offsets	varies
#5401-#5406	Current position (X, Y, Z, A, B, C)	50.325
#5410	Current tool number	5
#5420-#5428	Current tool offsets (H, D, etc.)	150.325
#_feed	Current feed rate	500.0
#_rpm	Current spindle speed	2000

Example - Reading current position:

```
#100 = #5421      (Store current X position in variable #100)
#101 = #5422      (Store current Y position)
#102 = #5423      (Store current Z position)
```

User Variables (Read-Write)

User-defined variables store values for calculations and parametric operations:

Local variables: #1 through #33 (cleared at program end)

```
#1 = 10.0          (Assign 10.0 to variable #1)
#2 = 20.5          (Assign 20.5 to variable #2)
#3 = [#1 + #2]     (Calculate: #3 = 30.5)
```

Global variables: #100 through #999 (persistent across programs)

```
#100 = 5           (Remains in memory after M30)
#101 = [#100 * 2]  (Calculate: #101 = 10)
```

Common variables: #500 through #999 (shared between programs, persistent)

```
#500 = 25          (Part count - persists across program runs)
```

Variable Assignment

Direct assignment:

```
#1 = 10.5      (Assign literal value)
#2 = #1        (Copy from another variable)
#3 = #5421     (Copy from system variable)
```

Expression assignment:

```
#4 = [#1 + #2] (Addition)
#5 = [#1 * #2] (Multiplication)
#6 = [#2 - #1] (Subtraction)
#7 = [#2 / #1] (Division)
```

Using Variables in Motion Commands

Variables can substitute for any numeric value:

```
#1 = 50.0      (X target position)
#2 = 25.0      (Y target position)
#3 = -10.0     (Z depth)
#4 = 500.0     (Feed rate)
```

```
G01 X#1 Y#2 Z#3 F#4 (Move to variable positions)
```

Result:

```
G01 X50.0 Y25.0 Z-10.0 F500.0
```

Mathematical Expressions

Arithmetic Operators

Operator	Function	Example	Result
+	Addition	[10 + 5]	15
-	Subtraction	[10 - 5]	5
*	Multiplication	[10 * 5]	50
/	Division	[10 / 5]	2
MOD	Modulo	[10 MOD 3]	1

Example:

```
#1 = [100 + 50]  (#1 = 150)
#2 = [#1 * 2]    (#2 = 300)
#3 = [#2 / 10]   (#3 = 30)
```

Trigonometric Functions

Function	Description	Example
SIN []	Sine (degrees)	SIN[30] = 0.5
COS []	Cosine (degrees)	COS[60] = 0.5
TAN []	Tangent (degrees)	TAN[45] = 1.0
ASIN []	Arc sine	ASIN[0.5] = 30
ACOS []	Arc cosine	ACOS[0.5] = 60
ATAN []	Arc tangent (2-argument)	ATAN[1]/[1] = 45

Example - Bolt circle calculations:

```
#1 = 100.0          (Bolt circle diameter)
#2 = 8              (Number of holes)
#3 = 360.0 / #2     (Angle between holes: 45 deg)

#10 = [#1/2] * COS[#3 * 1]  (X position, hole 1)
#11 = [#1/2] * SIN[#3 * 1]  (Y position, hole 1)

G81 X#10 Y#11 Z-20 R5 F100  (Drill hole 1)
```

Other Mathematical Functions

Function	Description	Example
SQRT []	Square root	SQRT[25] = 5
ABS []	Absolute value	ABS[-10] = 10
ROUND []	Round to nearest integer	ROUND[10.6] = 11
FIX []	Truncate (round down)	FIX[10.9] = 10
FUP []	Round up	FUP[10.1] = 11
LN []	Natural logarithm	LN[2.718] = 1
EXP []	Exponential (e^x)	EXP[1] = 2.718

Example - Calculate hypotenuse:

```
#1 = 30.0          (Side A)
#2 = 40.0          (Side B)
#3 = SQRT[#1 * #1] + [#2 * #2]  (#3 = 50.0)
```

Conditional Logic

IF-THEN-ELSE Statements

Syntax (FANUC/LinuxCNC style):

```
0100 IF [condition]
      (statements if true)
0100 ELSE
      (statements if false)
0100 ENDIF
```

Comparison operators: - **EQ**: Equal to - **NE**: Not equal to - **GT**: Greater than - **GE**: Greater than or equal to - **LT**: Less than - **LE**: Less than or equal to

Example 1: Simple condition

```
#1 = 10.0
#2 = 20.0

0100 IF [#1 LT #2]
      (MSG, #1 is less than #2)
      #3 = #1          (Use smaller value)
0100 ELSE
      #3 = #2
0100 ENDIF
```

Example 2: Check for zero to avoid division error

```
#1 = 100.0
#2 = 0
```

```
O200 IF [#2 EQ 0]
  (ABORT, DIVISION BY ZERO ERROR)
O200 ELSE
  #3 = [#1 / #2]
O200 ENDIF
```

Example 3: Adaptive depth based on tool diameter

```
#1 = 12.0          (Tool diameter)

O300 IF [#1 GT 10.0]
  #2 = 5.0          (Large tool: 5mm depth per pass)
O300 ELSE
  #2 = 2.0          (Small tool: 2mm depth per pass)
O300 ENDIF

G01 Z[0 - #2] F100  (Plunge to calculated depth)
```

Nested Conditions

```
#1 = 15.0          (Tool diameter)

O100 IF [#1 GT 20.0]
  #2 = 10.0         (Very large tool)
O100 ELSE
  O110 IF [#1 GT 10.0]
    #2 = 5.0         (Large tool)
  O110 ELSE
    #2 = 2.0         (Small tool)
  O110 ENDIF
O100 ENDIF
```

Loops and Iteration

WHILE Loop

Syntax:

```
O100 WHILE [condition]
  (statements)
O100 ENDWHILE
```

Example - Drill holes in a line:

```
#1 = 0              (Starting X position)
#2 = 100            (Ending X position)
#3 = 10             (Spacing between holes)

O100 WHILE [#1 LE #2]
  G81 X#1 Y0 Z-20 R5 F100  (Drill hole)
  #1 = [#1 + #3]          (Increment position)
O100 ENDWHILE

G80                  (Cancel cycle)
```

Result: Drills holes at X0, X10, X20, ... X100

DO-WHILE Loop

Syntax:

```
O100 DO [#var = start, end, increment]
    (statements)
O100 ENDDO
```

Example - Pocket with multiple depth passes:

```
#1 = 0                (Start depth)
#2 = -15              (Final depth)
#3 = -3               (Depth increment)

O100 DO [#1 = #3, #2, #3]    (Loop from -3 to -15, step -3)
    G01 Z#1 F100            (Plunge to depth)
    (... pocket toolpath at this depth ...)
    G00 Z5.0                (Retract)
O100 ENDDO
```

Result: Pockets at Z=-3, Z=-6, Z=-9, Z=-12, Z=-15

FOR Loop Style (Alternative Syntax)

Some controls use different syntax:

```
FOR #1 = 1 TO 10 STEP 1
    G81 X[#1 * 10] Y0 Z-20 R5 F100
ENDFOR
```

Subprograms with Parameters

Calling Subprograms with Arguments

Main program:

```
O1000                (Main program)
G54
T01 M06
G43 H01
S2000 M03
M08

(Call subprogram with different parameters)
#1 = 10.0 #2 = 10.0 #3 = -5.0    (X, Y, Z for pocket 1)
M98 P2000                    (Call pocket subprogram)

#1 = 50.0 #2 = 50.0 #3 = -10.0   (X, Y, Z for pocket 2)
M98 P2000                    (Call pocket subprogram)

M09
M05
M30
```

Subprogram (O2000 - parametric pocket):

```

O2000          (Pocket subprogram)
G00 X#1 Y#2    (Position to pocket center)
G00 Z5.0
G01 Z#3 F100    (Plunge to specified depth)
(... pocket cutting pattern ...)
G00 Z5.0      (Retract)
M99           (Return to main program)

```

Subprogram with Local Variables

Avoid conflicts by using local variables (#1-#33):

```

O3000          (Subprogram: drill bolt circle)
(Expects: #1=center X, #2=center Y, #3=diameter, #4=holes)

#10 = #1       (Store inputs in local variables)
#11 = #2
#12 = #3
#13 = #4
#14 = 360.0 / #13 (Calculate angle increment)

G00 X#10 Y#11   (Move to center)

O100 D0 [#15 = 0, #13-1, 1] (Loop through holes)
  #16 = [#12/2] * COS[#14 * #15] (Calculate X offset)
  #17 = [#12/2] * SIN[#14 * #15] (Calculate Y offset)
  G81 X[#10 + #16] Y[#11 + #17] Z-20 R5 F100
O100 ENDDO

G80
M99

```

Practical Applications

Application 1: Parametric Bolt Circle

```

O1000          (Main program)
G21 G90 G17 G54
T01 M06
G43 H01
S2000 M03
M08

(Define bolt circle parameters)
#100 = 50.0    (Center X)
#101 = 50.0    (Center Y)
#102 = 80.0    (Bolt circle diameter)
#103 = 6       (Number of holes)
#104 = 0.0     (Starting angle)
#105 = -15.0   (Hole depth)

(Calculate angle increment)
#106 = 360.0 / #103

(Loop through holes)

```

```

0100 DO [#1 = 0, #103-1, 1]
    #110 = [#102/2] * COS[#104 + #106 * #1]    (X offset)
    #111 = [#102/2] * SIN[#104 + #106 * #1]    (Y offset)
    G81 X[#100 + #110] Y[#101 + #111] Z#105 R5.0 F100
0100 ENDDO

G80
M09
M05
M30

```

Application 2: Adaptive Roughing Based on Material

```

02000          (Adaptive roughing)

(Material codes: 1=Aluminum, 2=Steel, 3=Stainless)
#100 = 1        (Select material)

(Set parameters based on material)
0100 IF [#100 EQ 1]
    #101 = 5.0      (Depth per pass)
    #102 = 800      (Feed rate)
    #103 = 3000     (Spindle speed)
0100 ELSE
    0110 IF [#100 EQ 2]
        #101 = 3.0
        #102 = 400
        #103 = 2000
    0110 ELSE
        #101 = 2.0
        #102 = 200
        #103 = 1500
    0110 ENDIF
0100 ENDIF

(Apply parameters)
S#103 M03
G04 P2.0

(Roughing with calculated depth)
0200 DO [#1 = [0 - #101], -15.0, [0 - #101]]
    G01 Z#1 F100
    (... cutting moves at F#102 ...)
0200 ENDDO

M05
M30

```

Application 3: Part Count and Tool Life Tracking

```

03000          (Main program with tool life tracking)

(Global variables for tracking)
#500 = 0        (Part count - initialized once)
#501 = 0        (T01 usage count)

```

```

#502 = 500          (T01 tool life limit)

(Increment part count)
#500 = [#500 + 1]
(MSG, STARTING PART #500)

(Check tool life)
O100 IF [#501 GT #502]
    (ALARM, TOOL 1 HAS EXCEEDED LIFE LIMIT)
    M00          (Stop for tool replacement)
    #501 = 0      (Reset counter after replacement)
O100 ENDIF

T01 M06
G43 H01
S2000 M03

(... machining operations ...)

#501 = [#501 + 1]    (Increment tool usage)

M05
M30

```

Application 4: In-Process Probing and Measurement

```

O4000          (Probe and measure part thickness)

G54
T01 M06        (Touch probe)
G43 H01

(Probe top surface)
G30.1          (Store current position)
G38.2 Z-50 F50 (Probe down until contact)
#100 = #5063    (Store Z position of top surface)

(Probe bottom surface - assumes open bottom)
G00 Z5.0
G38.2 Z-100 F50 (Probe down to bottom)
#101 = #5063    (Store Z position of bottom)

(Calculate thickness)
#102 = [#100 - #101]

(Check tolerance)
O100 IF [ABS[#102 - 25.0] GT 0.5]    (Target 25mm +/- 0.5mm)
    (ALARM, PART THICKNESS OUT OF SPEC: #102 MM)
    M00
O100 ENDIF

(MSG, PART THICKNESS OK: #102 MM)
M30

```

Macro Programming Best Practices

1. Document Variable Usage

```
(VARIABLE DEFINITIONS)
(#100 = Pocket center X)
(#101 = Pocket center Y)
(#102 = Pocket width)
(#103 = Pocket height)
(#104 = Pocket depth)
(#110-#119 = Temporary calculation variables)
```

2. Use Meaningful Variable Numbers

```
(Poor practice)
#1 = 50
#2 = 25
#3 = #1 + #2

(Better practice - grouped by function)
#100 = 50          (Part dimensions #100-#109)
#101 = 25
#110 = #100 + #101 (Calculations #110-#119)
```

3. Initialize Variables

```
(Initialize all variables at program start)
#1 = 0
#2 = 0
#3 = 0
```

4. Add Error Checking

```
O100 IF [#1 EQ 0]
  (ABORT, PARAMETER #1 NOT SET)
O100 ENDIF

O200 IF [#2 LT 0]
  (ABORT, PARAMETER #2 MUST BE POSITIVE)
O200 ENDIF
```

5. Test with Simple Values First

Before complex calculations, test with known values:

```
#1 = 10.0          (Simple test value)
#2 = 20.0
#3 = [#1 + #2]     (Should be 30.0)
(MSG, TEST RESULT: #3) (Verify on screen)
M00
```

Key Takeaways

1. **Variables** (#1-#999) store values for parametric programming
2. **System variables** (#5xxx, #_xxx) provide access to machine state and position
3. **Mathematical expressions** support arithmetic, trigonometry, and advanced functions
4. **Conditional logic** (IF-THEN-ELSE) enables adaptive programming

5. **Loops** (WHILE, DO) automate repetitive operations
 6. **Subprograms with parameters** create reusable, flexible code modules
 7. **Practical applications** include bolt circles, adaptive feeds, tool life tracking, probing
 8. **Best practices:** Document variables, check for errors, test incrementally
 9. **Parametric programming** enables part families and adaptive machining
 10. **Advanced features** require thorough testing and simulation
-

Next: Section 15.10 - Control System Dialects

Previous: Section 15.8 - Post-Processing